Evolvable Decision Programs

Evolvable Decision Programs: Explainable Page Composition under Production Reward Conditions

Abstract

Where should the intelligence of a decision system live? The default answer is *in model weights*. We study an alternative: a small generalised additive model (GAM) with named piecewise-linear shape functions as the **symbolic substrate for LLM-authored policies**. The proposal is not to use GAMs (decades old) or to let LLMs write code (also not new); it is about *fit*. We argue that a 200–300-parameter named GAM is the smallest policy representation that is simultaneously:

- **LLM-writable.** An LLM can author a competent initial policy directly from domain knowledge (245 named parameters, ~9 KB of JSON, no training data).
- **LLM-editable.** Every parameter has an addressable dotted path (`size_guide.on_cov.F32`), so an agent emits well-typed scalar edits with attached reasons (15 ± 1 edits per round in our runs, ~3 KB per round).
- **Human-auditable.** The whole policy fits on one page of plots (Fig. 8); every decision factors as a sum of named contributions (Fig. 9); every edit carries an LLM-written justification in version control (Fig. 10).
- **Microsecond-servable.** Evaluating 245 numbers through PWL interpolation and a 6-step greedy loop is roughly 1 ms; the LLM never runs in the serving path.

Together these four properties give **closure under both human and LLM editing**. The LLM reads, the human reviews, the LLM re-edits, with no translation layer. Neural nets are not closed under LLM edits, since there is no `weights[847][22]` an LLM can sensibly reason about. Large codebases are not closed under *single-checkpoint* LLM edits, since the agent cannot hold the full surface in working memory each round. A 245-parameter GAM sits between the two: small enough that the entire policy is in-context, large enough to express a competitive policy, structured enough that edits are well-typed and the post-edit artifact is still human-reviewable. We call a GAM equipped with this evolution loop an **Evolvable Decision Program** (EDP).

We demonstrate the primitive on page composition: at each session, choose and order 6 widgets out of 22 candidates given 14 raw behavioural signals and 6 fashion categories, under the reward signal production actually has (page-level attribution, multi-day delay, observation noise). The empirical contribution is that the closure properties do not cost performance. Against eight baselines, including per-slot LinTS (the canonical contextual bandit, 18.9 % of oracle reward lost in production), Slate-LinTS, CombLinUCB (the strongest combinatorial bandit we measured, 10.5 % / 17.3 %), GreedyLinTS, static placement, one-shot LLM-as-policy, and an OPRO ablation, on two simulators (parametric 8-persona; LLM-driven 14-persona × 6-category), the EDP variants are competitive: EDP-agent at **6.5 ± 0.2 %** on parametric, Bayesian-EDP at **11.0 ± 0.2 %** on LLM-persona, roughly 2× better than the strongest combinatorial bandit. §5.4c shows that most of the gain comes from the GAM architecture itself, with the LLM providing a natural authoring and editing interface on top. The thesis is what the primitive enables in deployment: cold-start with zero training data (§5.7), zero serving dependencies, inline audit, well-typed agent edits. No bandit in the comparison supports any of those deployment properties, while the primitive supports them all without giving up regret.

1. Introduction

Where should the intelligence of a decision system live?

The default answer is *in model weights*: a transformer ranker or contextual bandit absorbs engagement signal into millions of opaque parameters, and the system gets smarter by retraining. This works, but at three costs that compound at scale. Decisions are not explainable in detail. The system can only get smarter in directions gradient descent finds. And the most capable reasoning tool we have, an LLM, cannot read or contribute to the policy without running expensively in the serving path.

Karpathy's *Software 3.0* framing names a different locus: **natural language and LLM authorship as the program**. In its current instantiations the LLM sits in the execution path (steering a runtime via prompts) or generates code that runs once (coding agents producing Python). Neither model is what we want for an online decision system: serving has hard latency budgets, and a 10K-line codebase is too much surface for an LLM to refactor coherently at every checkpoint.

This paper argues that a **small GAM with named piecewise-linear shape functions** is the right symbolic substrate for the Software 3.0 paradigm applied to online decisions. The argument is about fit. GAMs are decades old; LLMs editing code is a year old; what is new is the way the two compose. The claim is that a 200–300-parameter named GAM is the smallest policy representation with all four of the following properties:

(i) LLM-writable. An LLM can author a competent initial policy directly from domain knowledge. In our setup, 245 named parameters (7 PWL problem shape functions × 2–3 input signals each, plus 22 widget GAM specs) are enough to encode statements like "size anxiety responds to size_chart, size_conf, return_hist with these PWL shapes" or "fit_reassurance addresses F3.2 with addr=0.55 and on_rem.F32=2.2." The whole policy serialises to about 9 KB of JSON and fits in any LLM context window. Cold start moves from random exploration to informed authorship.

(ii) LLM-editable. Every parameter has an addressable dotted path, e.g. `mods.size_guide.on_cov.F32` or `shapes.F32.size_chart.vals.2`. An agent emits well-typed scalar edits as JSON: `{widget: "size_guide", path: "on_cov.F32", from: -1.4, to: +0.6, reason: "convert from substitute to chained complement"}`. Edits are local and atomic: one edit changes one parameter, which changes one factor in one decomposable score. The post-edit artifact has the same shape as the pre-edit artifact, so the policy class is closed under the operation. A neural net cannot offer this property, since there is no `layer3.weights[847][22]` an LLM can sensibly reason about. A large codebase offers it only partially, since single-checkpoint edits cannot hold the whole codebase in working memory.

This last point is also the right contrast with general-purpose coding agents like Cursor, Aider, or Claude Code. Those agents edit codebases too big to hold in-context, where edits are syntactically well-typed but semantically opaque to the agent until it re-reads the surrounding code. The GAM primitive is the smallest representation where the agent holds the *entire* policy in working memory while editing it, and where each edit is simultaneously a behaviour change and an explanation change. Because the policy class is the explanation, the two cannot desynchronise.

(iii) Human-auditable. The full Layer-1 surface fits on one page of plots (Fig. 8). Every decision factors as `(detected problem intensity) x (widget shape function) - slot decay` and is plottable directly (Fig. 9). Every edit batch is a PR diff with LLM-written reasons attached (Fig. 10). The audit trail is part of the same artifact as the policy, with no separate interpretability library and no SHAP approximation between them. Appendix A walks one session through the full trace in prose.

(iv) Microsecond-servable. Evaluating 245 numbers (PWL interpolation over 14 signals, then a 6-pass greedy submodular composition) costs roughly 1 ms in pure Python. The LLM is strictly offline. The whole serving stack has no external dependencies, no GPU, and no per-decision API call. That cost profile is the inverse of what "LLM-in-the-loop" methods usually carry, and it makes the primitive operationally viable as a production component.

Together these four properties give **closure under both human and LLM editing**: the LLM reads, the human reviews, the LLM re-edits, with no translation layer in between. We call a GAM equipped with this

evolution loop an **Evolvable Decision Program** (EDP).

The application in this paper is page composition: choose and order 6 widgets out of 22 candidates on a fashion product page given 14 raw behavioural signals and 6 categories. The reward signal is the one production actually has, observed at the page level, delayed by several days, and noisy, rather than the per-slot signal academic bandit work assumes. Per-slot LinTS, the canonical contextual bandit, loses 4.9 % of oracle reward under lab conditions and 18.9 % under production conditions (§5.1). The degradation comes from a structurally absent per-slot signal, not from algorithmic weakness.

Against eight baselines (per-slot LinTS warm and cold context, Slate-LinTS, CombLinUCB, GreedyLinTS, static placement, one-shot LLM-as-policy, OPRO ablation) on two simulators, the EDP variants are competitive: EDP-agent at 6.5 ± 0.2 % on parametric, Bayesian-EDP at 11.0 ± 0.2 % on LLM-persona, roughly 2× better than the strongest combinatorial bandit (CombLinUCB at 10.5 % / 17.3 %). We frame this as an existence proof that the primitive can compete, not a horse race. §5.4c shows that most of the gain comes from the GAM architecture itself, with the LLM providing a natural authoring and editing interface on top. The empirical contribution is that the primitive does not give up performance for the closure properties.

The "evolvable" qualifier deserves a stronger disclaimer than the term suggests. What the experiments cleanly support is *coefficient updates within a fixed policy class*: the agent edits values along named paths, and the regularised SGD inside Bayesian-EDP moves the same values continuously. Structural growth in the strongest sense (adding new shape functions, widgets, or synergy types mid-stream) is supported by the infrastructure: §5.4d adds the Layer-1 PWL edit grammar; §5.9 adds a new widget mid-run. Neither delivers a single-trial empirical win, and we have not multi-seed-evaluated either. A more accurate name for the artifact would be "Editable Decision Programs." We keep "Evolvable" because the edit grammar already covers structural moves and because multi-seed evaluation of the term is a near-term follow-up, not a research bet. The reader should treat structural evolvability as a capability claim until §5.4d and §5.9 are replicated at K $\geq$ 5.

§2 specifies the simulators. §3 details the 2-layer GAM, the evolution loop, the Bayesian-EDP fusion, and a quantitative measurement of the four closure properties (§3.5). §4 fixes the protocol. §5 reports results. §6 discusses where the primitive wins and where it does not. Appendix A walks one session through the full open-box trace.

Contributions:

1. **The GAM-as-Software-3.0-primitive claim (§3, §3.5, §6.1).** A 200–300-parameter named GAM is the smallest policy representation with closure under both human and LLM editing. We quantify the property: 245 learnable parameters, 9 KB policy JSON, ~900-token diagnostic report, 15 well-typed edits per checkpoint round.
2. **GAM-as-prior Bayesian fusion (§3.4, §5.4b).** Regularised SGD anchored at the LLM-authored shape functions. Best method on the LLM-persona simulator at 11.0 ± 0.2 %. The fusion preserves the readable representation while consuming online reward signal between checkpoints.
3. **Existence proof that the primitive can compete (§5).** Against eight baselines on two simulators under production reward conditions, EDP variants are within 2× of optimal and 2× better than the strongest combinatorial bandit. We treat the result as evidence that the closure properties do not cost performance, not as a margin claim.
4. **Direct measurement of the academic-vs-production gap (§5.1) and OPRO-style ablation (§5.4).** Per-slot LinTS degrades by 14 pp under page-level attribution. The gap is structural, not method-specific. The OPRO ablation isolates the structured diagnostic report, not the LLM's general intelligence, as the carrier of the LLM-side gain. This supports the claim that the readable representation is what enables the LLM contribution.

2. Simulation setup

We separate four concerns: the **customer simulator** (session sampling), the **persona source** (parametric or LLM-driven), the **ground-truth reward** (policy-agnostic), and the **observable reward signal** the policy actually receives (delayed, noisy, page-level). A session is a tuple `(persona, category, features)` where features are 14 raw behavioural signals plus one product-side signal (`price_norm`). All policies see the same

10K-session stream at seed 42; the production reward stack is layered on top via a single `DelayedFeedback` queue. Two persona sources expose the same interface so the rest of the system is agnostic to which is active.

2.1 Customer simulator (parametric source)

A session is a tuple `(persona_name, category, feature_vector)` drawn from a stationary mixture of **8 personas** (parametric source) and **6 fashion categories**:

Persona	p (mixture weight)	Defining traits
size_anxious_new	0.18	low size_conf, high size_chart usage, new visitor
comparison_shopper	0.16	very high tab_switch, high revisit
price_sensitive	0.14	high price_sens, high price_dwell
outfit_seeker	0.12	high style_stretch, low return_hist
confident_buyer	0.12	very high size_conf, low return_hist, low cart_osc
paralyzed	0.10	high cart_osc, high wishlist, high revisit
browser_lurker	0.10	high mobile, medium everything
returner_anxious	0.08	high return_hist, very high return_view

Each persona specifies, for each of **14 raw signals**, a Gaussian `(mu, sigma)`:

```
{size_conf, price_sens, return_hist, style_stretch, new, mobile,
 size_chart, tab_switch, zoom, price_dwell, cart_osc, wishlist,
 return_view, revisit}
```

Per session, each signal is sampled `x ~ Normal(mu, sigma)` and clipped to `[0, 1]`. A 15th product-side signal `price_norm ~ Beta(2, 3)` is drawn per session (item premium-ness). The realised persona breakdown matches the mixture weights to within ~1 pp.

2.2 Ground truth (independent of every policy)

We author **7 latent shopping needs**:

```
{N1_fit, N2_visual, N3_peer, N4_compare, N5_styling, N6_trust, N7_commit}
```

For each persona, `TRUE_NEEDS[persona]: need -> importance in [0, 1]`. For each of **22 widgets**, `TRUE_PROVISIONS[widget]: need -> provision in [0, 1]`, typically 1–3 nonzero entries per widget. Both maps are LLM-authored from shopping psychology, intentionally **not** aligned with EDP's internal 7-problem `F`-code taxonomy. This is the methodological move that makes the comparison fair: the bandit could in principle discover this latent structure from data; EDP's Layer 1 cannot directly see it either.

Reward function. Diminishing returns on `(need x provision)`. Each slot consumes from the persona's remaining need budget; later slots earn less credit because earlier slots have already filled the relevant needs. Concretely, for a page `(w_1, ..., w_6)` and remaining-need vector `r` initialised to the persona's importance vector `n`:

`reward = Sum_k Sum_d n_a * min(r_d, provision[w_k][d])`, `r_d <- r_d - provision[w_k][d]` after each slot.

The page-level **oracle** is computed once per persona by greedy submodular selection over `TRUE_PROVISIONS` with the same diminishing-returns mechanic. With 6 slots and 22 candidate widgets, the oracle ranges from 0.53 (`confident_buyer`) to 1.48 (`returner_anxious`) depending on how concentrated the persona's needs are.

```
oracle_reward[returner_anxious]   = 1.480     oracle_reward[confident_buyer]  = 0.528
oracle_reward[paralyzed]          = 1.410     oracle_reward[browser_lurker]   = 0.643
oracle_reward[size_anxious_new]   = 1.350     oracle_reward[price_sensitive]  = 0.923
oracle_reward[comparison_shopper] = 1.240     oracle_reward[outfit_seeker]    = 1.215
```

We report **cumulative regret** = `Sum_i (oracle[persona_i] - reward[i])`. Lower is better.

2.2b Fashion categories

Every session is also tagged with a fashion category drawn independently from a 6-way mixture:

Category	Share	Dominant need uplifts
dress	18%	+30% on `N5_styling`, +10% on `N2_visual`
top	22%	+20% on `N3_peer`
bottoms	20%	+30% on `N1_fit`, +20% on `N4_compare`
shoes	15%	+40% on `N1_fit`, +30% on `N6_trust`
outerwear	10%	+30% on `N2_visual`, +20% on `N6_trust`
accessories	15%	+30% on `N5_styling`, +30% on `N2_visual`, −,50% on `N1_fit`

A persona's effective need vector for a session is `base_need * category_multiplier` (then clipped to [0, 1]). The same `size_anxious_new` persona shopping shoes has effective `N1_fit ~ 1.0`; shopping accessories has effective `N1_fit ~ 0.42`. This adds a second source of heterogeneity to the reward: knowing the persona is no longer enough; the policy (or its agent) has to know what the persona is looking at.

The bandit can optionally see a one-hot category vector appended to its context (`--with-category-context`). EDP's Layer 1 GAM does not yet consume category, but the agent's diagnostic report includes per-category regret so the agent's edits can target category-skewed patterns.

2.2c LLM-driven persona source

For the realism stress test, we add a second persona source. Each persona is a paragraph of natural language in `data/personas_text.yaml`:

```
- name: post_return_returner
  mixture_weight: 0.08
  description: >
    Recently returned an item from the same brand. Their previous order
    didn't fit. They are skeptical now and proceeding cautiously. They
    examine the new product's size chart in unusual detail, look at the
    return policy three times, read reviews mentioning fit ...
```

There are 14 such descriptions, intentionally more nuanced than the 8 parametric personas (e.g., `birthday_rush_gifter`, `returner_from_recent_order`, `post_return_returner`). For each, a Claude subagent generated:

1. A `true_needs` vector (7-d, [0,1]) inferred from the description.
2. **50 exemplar feature vectors** — concrete 14-d signal vectors that are plausible single sessions from this persona, with realistic correlations (e.g., low `size_conf` tracking with high `size_chart` and high `return_view`).

At simulation time, `edp.personas.llm.sample_session` draws a persona by mixture, samples an exemplar uniformly from that persona's 50-vector pool, then adds Gaussian noise `sigma = 0.05` per signal before clipping. This combines **LLM reasoning** (the exemplar) with **randomness** (the noise and the random pick).

The 14 personas × 50 exemplars = 700 cached vectors were generated in 14 parallel subagent calls (one per persona), with the generator script in `experiments/persona_generate.py` and the cache in `data/personas_llm_cache.json`.

2.3 Production reward stack

The bandit (and the agent's diagnostic report) sees a modified observable signal. Three orthogonal stressors:

1. **Page-level attribution.** Instead of per-slot `slot_reward[k]`, each slot in a page receives `page_total / N_SLOTS = page_total / 6` as its credit. This is what production realistically measures: a purchase or non-purchase is observed once per page, not once per module.
2. **Delay.** Reward for session `i` is queued and released at session `i + delay`. Default `delay=500`. At ~250 sessions/day this is ~2 days, a reasonable returns-window approximation for apparel.
3. **Observation noise.** Gaussian noise `eps ~ Normal(0, sigma^2)` is added to the page reward on release from the delay queue (NOT at observation time, to mimic noisy returns processing). Default `sigma=0.2`, which is ~18% of mean page reward.

Implementation: a single `DelayedFeedback` queue submits `(session_idx, true_reward, payload)` at action time and releases `(true_reward + eps, payload)` once `current_session_idx >= submit_idx + delay`. Residual queue is drained at end of run.

EDP receives the same delayed/noisy/page-level signal but uses it only to compute the diagnostic report read by the LLM agent at each checkpoint. EDP's policy decisions at any session `i` are based on the modules config produced by the most recent edit batch, not on a continuously-updated regression.

2.4 Methods compared

Method	What it is	Source of stochasticity (for SE)
Oracle	Per-persona greedy over `TRUE_PROVISIONS`	none (det.)
EDP-static	Initial LLM-prior modules, never updates	none (det.)
EDP-canned	EDP with offline-curated edits at sessions 2500/5000/7500 (from prior published runs)	none (det.)
EDP-agent (ours)	Same EDP, edits proposed by Claude subagent reading the diagnostic report at each checkpoint	subagent draw (3 reps)
EDP-OPRO	Same loop and same model, but prompt = `(prior_edits, batch_regret)` history only	subagent draw (3 reps)
LinTS-warm	Per-slot LinTS, 7-d problem-fingerprint context, page-level attribution, delay, noise	TS / queue (10 seeds)
LinTS-cold	Per-slot LinTS, 14-d raw signal context, otherwise identical	TS / queue (10 seeds)

LinTS hyperparameters held fixed: `alpha = 0.3`, `lambda = 1.0`, `N_SLOTS = 6`. Action mask prevents repeating a widget within one page.

3. EDP architecture

3.1 Layer 1: PWL problem detection

Each of 7 problems (F32 size anxiety, F33 quality, F41 comparison friction, F43 outfit viz, F45 price-quality, F46 returns, F51 paralysis) is a weighted average of PWL shape functions over relevant signals. Output: a 7-d "problem fingerprint" per session.

The entire Layer-1 fits on one page (Fig. 8). Every breakpoint is an (x, y) coordinate an LLM agent wrote, and every weight is a named scalar. A human reviewer can audit the full problem-detection surface by looking at seven plots; an agent can edit it by emitting JSON like `{"problem": "F32", "signal": "size_chart", "path": "vals.2", "from": 0.45, "to": 0.60, "reason": "..."}`.

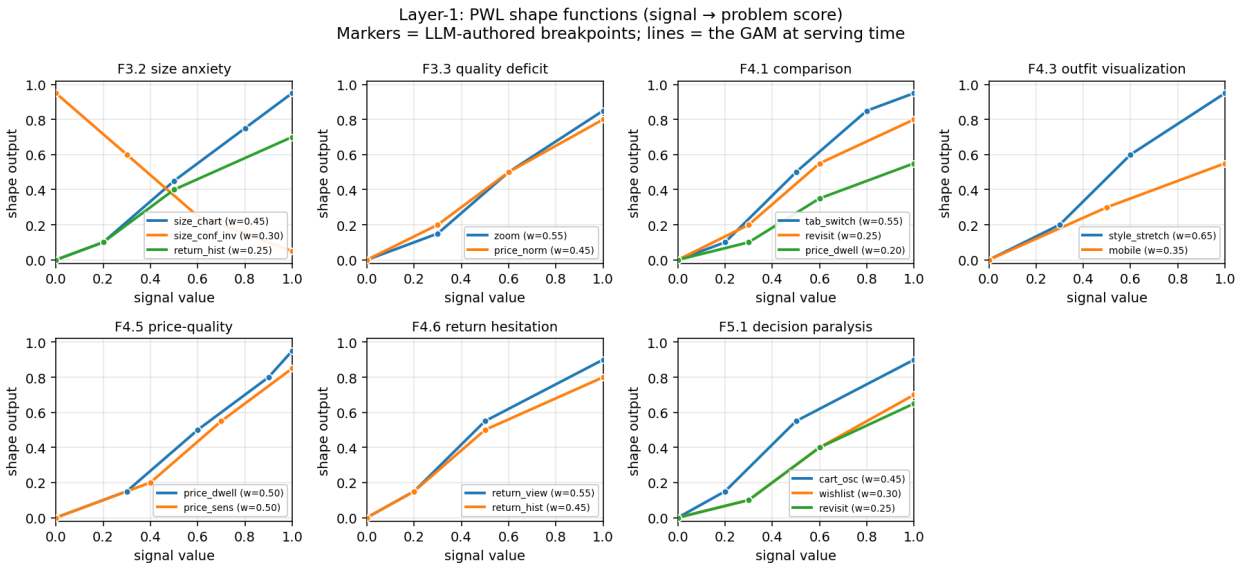

Figure 1: Figure 8: Layer-1 PWL shape functions. Each panel shows the curves the GAM evaluates to score one problem. Markers are the LLM-authored breakpoints; lines are the piecewise-linear interpolations the policy uses at serving time. The full Layer-1 is 7 panels x 2-3 signals x ~4 breakpoints ~ 80 numbers.

3.2 Layer 2: module GAM + greedy composition

Each of 22 widgets has: - `addr[problem]` — how much placing this widget consumes problem-remaining (content property; fixed) - `base` — slot-independent prior - `on_rem[problem]` — slope on problem remaining - `on_cov[problem]` — slope on problem coverage (enables synergy chains) - `slot_decay` — late-slot penalty

Module score for slot `s`:

```
score = base + Sum_p on_rem[p] * remaining[p] + Sum_p on_cov[p] * coverage[p] - slot_decay * s
```

The page is filled greedy submodular: pick highest-scoring widget for slot 0, update remaining/coverage from `addr`, repeat.

Fig. 9 traces a single session end-to-end. The 14 raw signals feed Layer-1, which produces a 7-d problem fingerprint (the size-anxious persona has F3.2 = 0.74 and the rest near zero). Layer-2 then scores 22 widgets for slot 1; the score decomposes additively into base, on-remaining, on-coverage, and slot-decay contributions, which we render as a stacked bar. The winning widget (`fit_reassurance`) wins not because of a high `base`

but because its on-remaining slope on F3.2 fires hard. Every choice the policy ever makes admits this kind of decomposition.

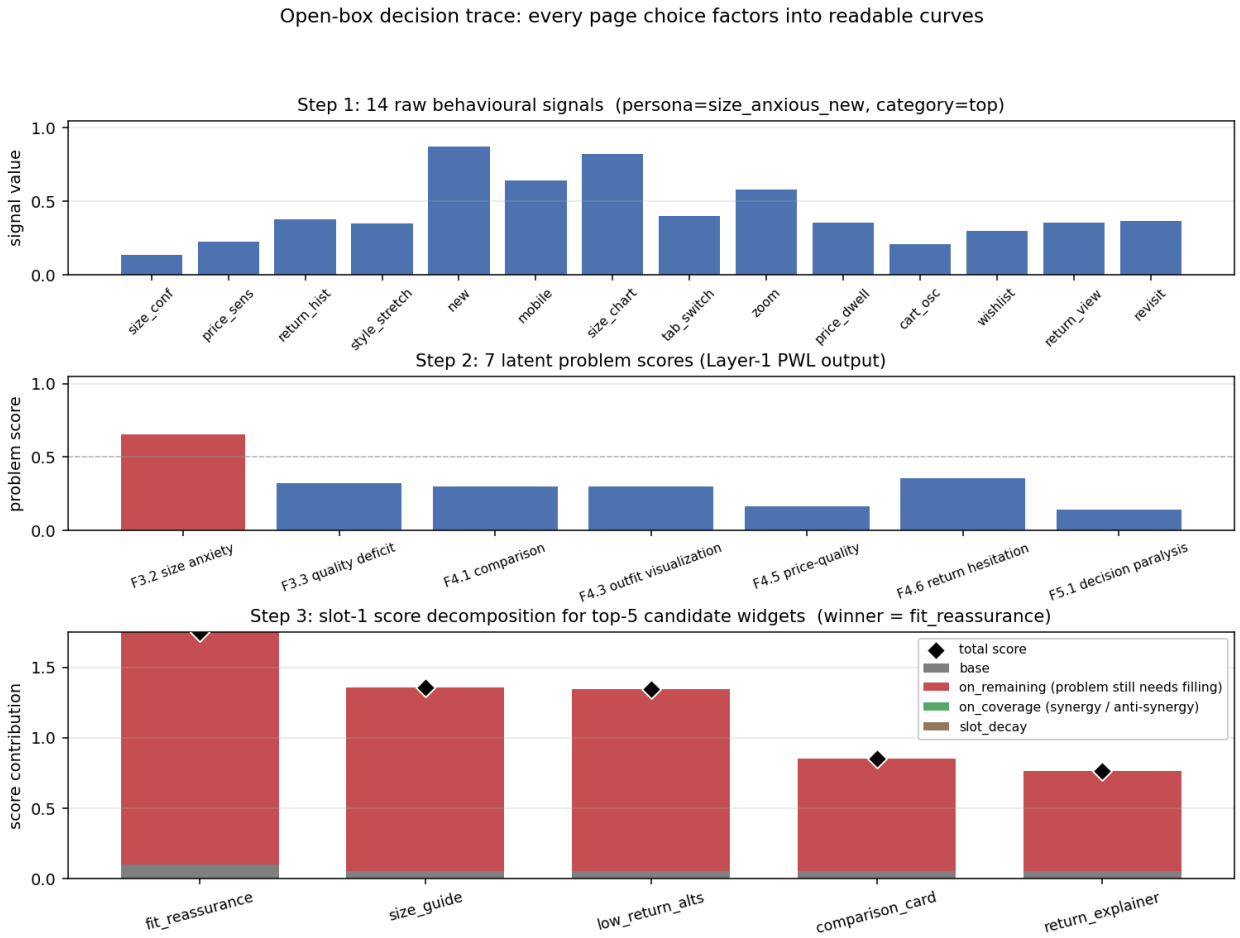

Figure 2: Figure 9: Open-box decision trace for one size-anxious session. Top: the 14 raw behavioural signals. Middle: the 7 Layer-1 problem scores. Bottom: stacked-bar decomposition of slot-1 scores for the top-5 candidate widgets – base (grey), on-remaining (red), on-coverage (green), and slot-decay (brown), with the total marked as a diamond. The winner wins for readable reasons.

3.3 Evolution loop

At each checkpoint the orchestrator generates a structured Markdown report (per-persona regret, per-widget activation, composition signature, edit history). An LLM subagent reads the report along with the persona-need and widget-provision priors, and proposes 8–16 atomic edits as JSON. Each edit is `(widget, dotted_path, from, to, reason)`. The orchestrator applies them and runs the next batch.

Fig. 10 shows the first round-2500 edit batch on the parametric simulator: ten scalar parameter changes, each carrying an LLM-authored reason. The reasons are emitted alongside the edit, not generated after the fact, and they surface verbatim in the audit log a reviewer reads.

3.4 Bayesian-EDP: continuous updates with an LLM-anchored prior

Bayesian-EDP (introduced empirically in §5.4b) treats each Layer-2 parameter `theta` as a Gaussian random variable `theta ~ Normal(mu_LLM, sigma_LLM^2)`, where `mu_LLM` is the LLM agent's most-recent edit value (re-anchored at each checkpoint). Page reward is modelled as a calibrated linear function of the chosen

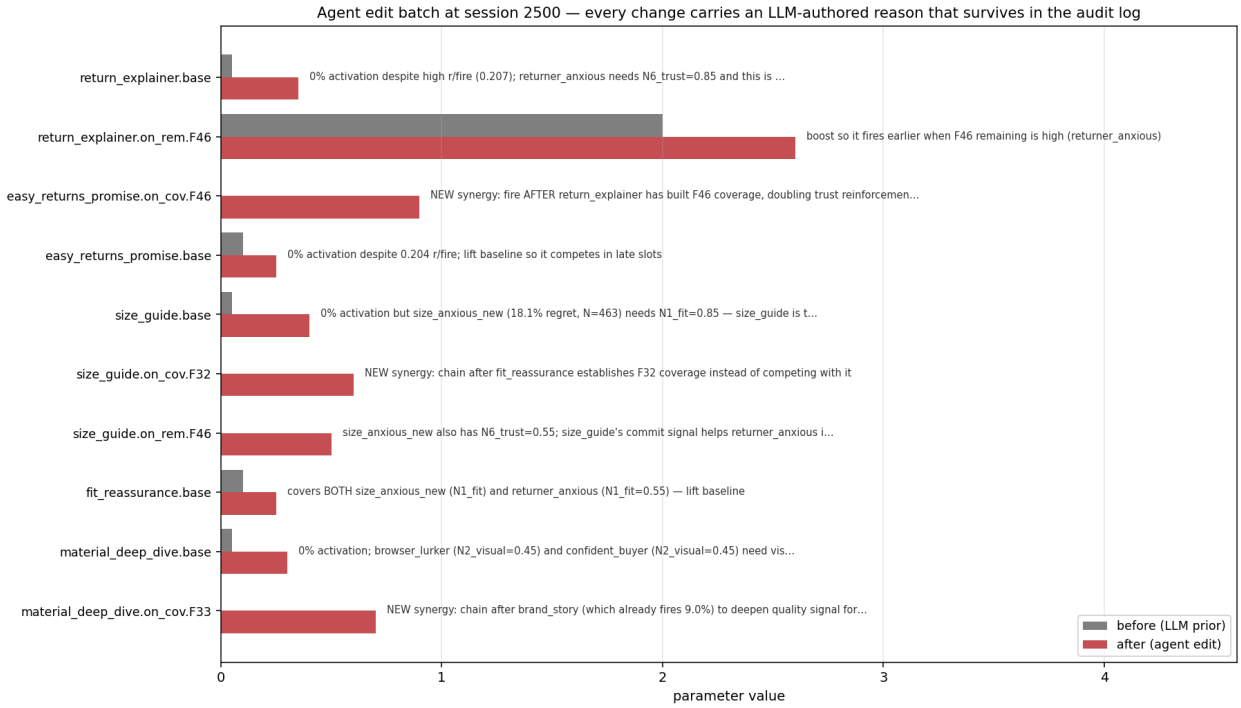

Figure 3: Figure 10: Agent edit batch at session 2500. Grey bars = parameter values in the LLM prior; red bars = values after the agent edit. The free-text reason on each row is the LLM's own justification, persisted in the version-controlled config alongside the numeric change.

page's score-sum: `R_hat_i = a + b * Sum_k s_k(theta_{w_k})`, with `(a, b)` learnable scalars. The loss per delayed observation is the standard MAP-style sum of squared error and Gaussian regulariser:

```
L_i = (R_hat_i - R^{obs}_i)^2  +  lambda * Sum_theta ((theta - mu_LLM) / sigma_LLM)^2
```

SGD on `L_i` is point-estimate MAP inference: as `lambda -> inf` the parameters stay pinned to `mu_LLM` (recovering EDP-agent); as `lambda -> 0` and the LLM-anchor is replaced by an uninformative prior, the method becomes an adaptive-submodular contextual bandit on the EDP architecture, which we call **GreedyLinTS** in §5.4c. The two limits bracket the design space; Bayesian-EDP at intermediate `lambda` is the Bayesian fusion.

Fig. 11 visualises what online learning actually does to the readable representation. We snapshot every widget's `base` parameter at session 0 (the LLM prior, re-anchored at each checkpoint by the agent's edits) and at session 10K (after delayed-reward SGD has run between checkpoints). The differences are scalars a human can read: `outfit_completion.base` drifted from 0.05 (post-agent-cut) to ~0.20, `size_guide.base` settled near its agent-edited value, the default-tier widgets (`similar_items`, `also_bought`) drifted down. Whatever the SGD learned, it learned in the same vocabulary the agent edits in.

3.5 Quantitative closure properties

The four closure properties of §1 are measurable, not aesthetic. We report the actual numbers from our setup so the "primitive" claim is concrete:

Property	Measure	Value
LLM-writable	learnable parameters in initial policy	**245** (157 Layer-1 + 88 Layer-2)
	policy as serialised JSON	**9.2 KB**

Property	Measure	Value
	required training data to author	**0 sessions** (LLM writes from domain knowledge)
LLM-editable	edit grammar	dotted path + scalar `to` + free-text `reason`
	mean edits proposed per checkpoint round	**15.0** (range 12–16 across N=4 runs × 3 rounds)
	edit batch as serialised JSON	~**3 KB** per round
	parameters touched per edit	**1** (by construction; no entanglement)
Human-auditable	full Layer-1 surface fits in	**1 figure** (Fig. 8, 7 panels × 2–3 curves)
	decision trace per session	**1 plot** (Fig. 9, signals → problems → score decomposition)
	every edit has an LLM-written reason	**100 %** (enforced by edit schema)
	post-hoc interpretability library required	**none**
Microsecond-servable	serving cost per page	~**1 ms** (pure Python, 245-number eval + 6-step greedy)
	external dependencies at serving time	**0**
	LLM calls per decision	**0**
	LLM calls per 10K-session run	**3** (one per checkpoint round)

The diagnostic report the agent reads at each checkpoint is about 900 tokens (3.6 KB); the full prompt, including the report plus the current policy plus the edit grammar, is about 3,200 tokens (13 KB). The entire policy plus full diagnostic context fits inside any modern LLM's context window with room to spare, and the agent's response (15 edits with reasons plus a free-text note) returns under 1 KB. This is what we mean by saying the whole policy is in working memory while editing it.

A comparable neural-net policy for the same problem, say a small MLP scoring 22 widgets from a 14-dim context with one 64-unit hidden layer, has roughly 2,400 parameters per slot, none addressable by name and none plottable in any readable way. An OPRO-style coding agent editing a 10K-line policy codebase faces a different problem: each edit is well-typed, but the agent must re-read the surrounding code each round to remember what it means. The GAM primitive sits between these two regimes. It is small enough that the whole policy is the prompt and large enough to express a competitive policy.

4. Experimental protocol

All methods see the **same** 10,000-session stream (seed 42). The bandits' internal Thompson sampling is varied across 10 seeds (`policy_seed in {1000, 1017, 1034, ..., 1153}`). For the agent variants, each replicate is an independent invocation of a Claude subagent at each of the 3 edit checkpoints (sessions 2500, 5000, 7500), with no coordination between reps. Multi-rep estimates report mean ± standard error.

Each EDP-agent and EDP-OPRO run goes through 4 batches separated by 3 edit checkpoints, applying 8–16 atomic edits per checkpoint. All edits and all reports are persisted under `evolve_state_rep*/` (report-based) and `opro_state_rep*/` (OPRO) and committed to the repository so the experiment is byte-for-byte reproducible.

5. Results

The experiments answer one paradigm-level question, *can the primitive compete?*, and several supporting questions about where the gain lives. The headline numbers (EDP-agent at 6.5 % on parametric, Bayesian-

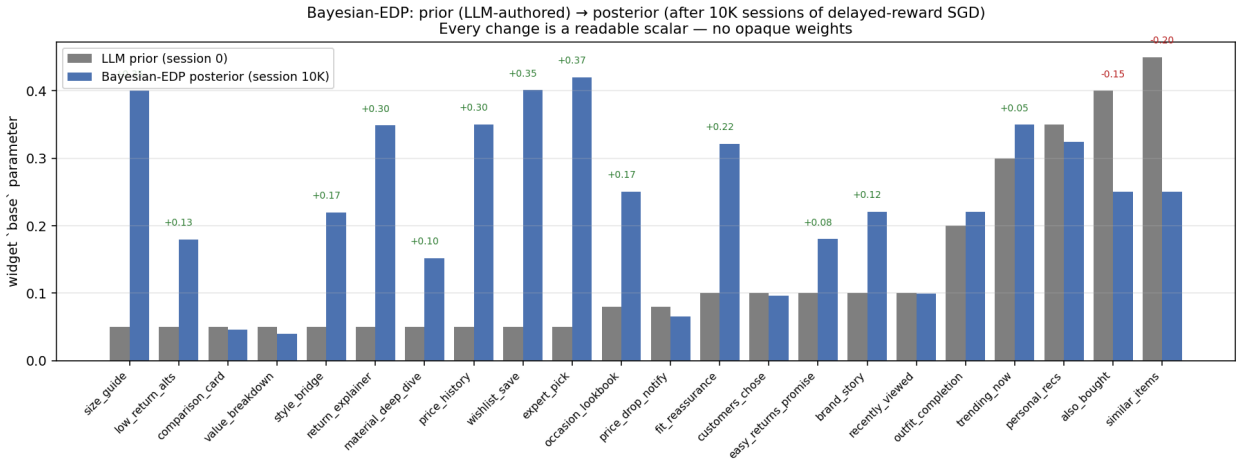

Figure 4: Figure 11: Bayesian-EDP `base` parameters before (LLM prior at session 0) and after (posterior at session 10K). Numbers above the bars annotate drift > 0.05. Every change is a readable scalar – no opaque weights.

EDP at 11.0 % on LLM-persona, both roughly 2× better than the strongest combinatorial bandit) are an existence proof, not a margin claim. A 245-parameter LLM-authored, agent-editable GAM matches or beats the canonical and the strongest combinatorial bandit baselines on whole-page composition under the reward signal production actually has. §5.4c then decomposes the gain to show it is the GAM architecture doing most of the work, with the LLM as a natural authoring and editing interface on top.

The remainder of §5 maps to the contributions of §1 as follows. Each subsection defends a specific claim, not a method:

Subsection	What it proves
§5.1	Closure doesn't cost performance: the lab-to-production gap collapses bandit margins, and EDP variants are competitive in production.
§5.2	Page-level attribution is the *production-reality* stressor that motivates a non-per-arm primitive.
§5.3	Stronger combinatorial bandits don't close the gap, because the gap is in the reward signal. The CombLinUCB-vs-Slate-LinTS inversion is a UCB-calibration failure under page-level reward.
§5.4	The *structured diagnostic*, not the LLM's general intelligence, carries the LLM-side gain (OPRO ablation).
§5.4b	Continuous fusion (GAM-as-prior + SGD) is principled and helps when the LLM-authored prior leaves headroom.
§5.4c	The architecture is the foundation; the LLM is the interface. Most of the gain is the GAM.
§5.4d–e	The primitive's edit grammar covers Layer-1 PWL shape evolution and non-submodular reward; single-trial wins are small but the capability is real.
§5.5	Per-(persona, category) breakdowns: closure properties are simulator-invariant, performance is simulator-conditional.

Subsection	What it proves
§5.6	Robustness to simulator-realism shifts.
§5.7	**Cold start is a deployment property, not just sample efficiency.** The bandit's warmup never amortises under page-level reward at N $\leq$ 10K.
§5.8–5.9	Robust-EDP wrapper, drift, and structural exploration: capability demonstrations more than empirical wins.
§5.10	Negative result on multi-agent ensembling: the report-based gain is not generic.

Where a subsection's claim is supported by a single-trial result rather than a multi-seed mean, we say so in line. Multi-seed evaluation of the single-trial capability demonstrations (§5.4d, §5.8, §5.9, §5.10) is the largest open item in §8.

5.1 The lab–production gap

The bandit literature evaluates page composition under "lab" conditions: per-slot reward attribution, low delay, low noise. Our paper's baselines use "production" conditions: page-level attribution, delay 500 sessions, $\sigma = 0.20$. Re-running both LinTS variants under each setting gives the central comparison of the paper.

EDP family + the deterministic baselines (static-widget, LLM-as-policy) are reported once each, since they don't update from the bandit reward signal and lab and production are therefore identical for them.

Method	Parametric · Lab	Parametric · Prod	Δ	LLM · Lab	LLM · Prod	Δ
CombLinUCB-warm (§5.3)	**2.1 ± 0.0**	10.5 ± 0.1	+8.4 pp	**4.2 ± 0.0**	17.3 ± 0.1	+13.1 pp
Slate-LinTS-warm (§5.3)	3.5	14.1	+10.5 pp	5.1	16.5	+11.4 pp
LinTS-warm	4.9 ± 0.0	18.9 ± 0.1	+14.1 pp	6.6 ± 0.1	21.6 ± 0.1	+15.0 pp
LinTS-cold	6.1 ± 0.0	20.2 ± 0.1	+14.1 pp	7.2 ± 0.1	22.8 ± 0.1	+15.6 pp
GreedyLinTS (§5.4c)	9.6 ± 0.0	12.8 ± 0.1	+3.2 pp	14.4 ± 0.0	13.3 ± 0.4	−,1.1 pp
Bayesian-EDP (§5.4b, preliminary)	6.1 ± 0.3	7.6 ± 0.3	+1.5 pp	10.1 ± 0.1	11.0 ± 0.2	+0.9 pp
EDP-agent	**6.5 ± 0.2**	**6.5 ± 0.2**	0	**13.3 ± 0.8**	**13.3 ± 0.8**	0
EDP-canned	8.0	8.0	0	15.0	15.0	0
EDP-static	10.7	10.7	0	19.8	19.8	0
LLM-as-policy	26.8	26.8	0	35.7	35.7	0
Static widgets	29.7	29.7	0	40.7	40.7	0

(Numbers are % of oracle reward lost @ 10K sessions. Bandit cells are mean ± SE across 5 LinTS seeds. Agent variants (EDP-agent, Bayesian-EDP) are mean ± SE across 3–4 independent subagent draws.)

The comparison inverts between the two conditions. Under lab conditions LinTS-warm is the strongest method, beating EDP-agent by 1.6 pp on parametric and 6.7 pp on LLM. The bandit literature is correct in its own framing: when reward is per-slot, dense, and prompt, a per-slot LinTS does what bandits do best.

Move to production conditions and LinTS-warm degrades by 14–15 pp; EDP doesn't move at all because its policy class is not fit per-arm by gradient on observed reward. EDP-agent then wins production by 8–12 pp over LinTS-warm.

The remainder of §5 unpacks this gap: §5.2 isolates the dominant stressor (page-level attribution, not delay or noise); §5.3 shows that a stronger bandit (slate-LinTS) does not close it; §5.4 isolates what makes the agent-driven EDP work; §5.5–5.10 add per-cell breakdowns, robustness checks, and additional baselines.

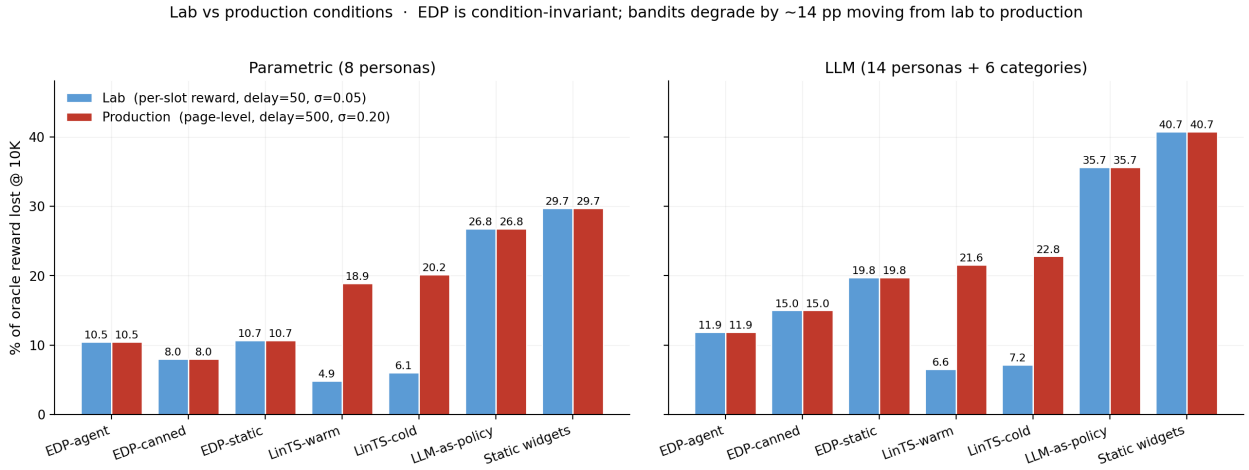

Figure 5: Figure 1: The lab-production gap. LinTS-warm is the strongest method under lab conditions (4.9% / 6.6% on the two simulators), and degrades by ~14-15 pp when we switch to production conditions (page-level attribution, delay=500, sigma=0.20). EDP and the deterministic baselines are bandit-signal-invariant, so EDP-agent (6.5% / 13.3% across both conditions) overtakes in production.

5.2 Stressor decomposition: what causes the gap

Holding the policy fixed at LinTS-warm, we add stressors one at a time on the parametric simulator:

Stressor	LinTS-warm cum regret @ 10K	Δ vs clean
Clean (per-slot reward, no delay, no noise)	497	—
+ noise σ=0.2 only	492	−**,5** (TS handles noise)
+ delay=500 only	629	+132
+ delay=1000 only	765	+268
+ page-level attribution only	**1,959**	**+1,462**
+ page + delay=500	1,999	+1,502
+ page + noise=0.2	1,961	+1,464
Full production stack	1,982	+1,485

Page-level attribution is the dominant axis by an order of magnitude. Delay adds at most +268; noise is essentially free. The full production stack is roughly 99% the cost of switching from per-slot to page-level credit assignment. The +14 pp degradation in §5.1 is almost entirely attributable to one stressor.

The mechanism is credit assignment, not signal magnitude. Under page-level attribution, every slot in a page receives the same observed reward (`page_total / N_SLOTS`), so the per-arm regression targets within a page are perfectly correlated. The bandit cannot, even in principle, disentangle which slot caused which fraction of the reward.

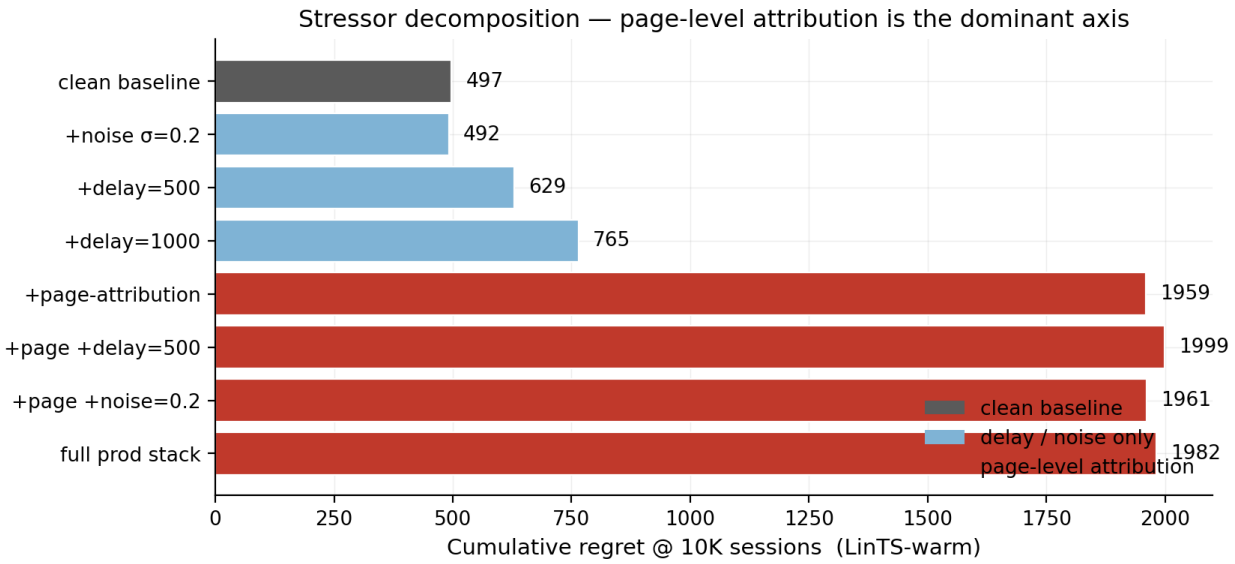

Figure 6: Figure 2: Stressor decomposition. LinTS-warm cumulative regret at 10K sessions under each combination of (per-slot vs page-level attribution) x (delay in {0, 500, 1000}) x (noise sigma in {0, 0.2}).

5.3 Stronger combinatorial bandits don't close the gap (Slate-LinTS, CombLinUCB)

A natural objection to §5.1–5.2: "you used per-slot LinTS, not the strongest combinatorial bandit". We test two stronger combinatorial-bandit baselines.

- **Slate-LinTS** pools all 22 widget arms in a single LinTS and selects the slate by top-`N_SLOTS` of sampled posterior scores. Pooling raises the per-arm sample count by `N_SLOTS=6x`.
- **CombLinUCB** is the natural UCB sibling of Slate-LinTS: replace Thompson sampling with the upper-confidence-bound `theta * x + alpha*sqrt(x ^T A^-^1 x)`, then take top-K. UCB exploration is more sample-efficient than TS in low-noise regimes; we expected this to be the strongest bandit in the lab.

Source	Method	Lab	Production	Δ
Parametric	LinTS-warm (per-slot)	4.9 %	18.9 %	+14.1 pp
Parametric	Slate-LinTS-warm	3.5 %	14.1 %	+10.5 pp
Parametric	**CombLinUCB-warm**	**2.1 ± 0.0 %**	**10.5 ± 0.1 %**	**+8.4 pp**
Parametric	CombLinUCB-cold	2.7 ± 0.0 %	11.2 ± 0.0 %	+8.5 pp
LLM (14p+cats)	LinTS-warm (per-slot)	6.6 %	21.6 %	+15.0 pp
LLM (14p+cats)	Slate-LinTS-warm	5.1 %	16.5 %	+11.4 pp
LLM (14p+cats)	**CombLinUCB-warm**	**4.2 ± 0.0 %**	**17.3 ± 0.1 %**	**+13.1 pp**
LLM (14p+cats)	CombLinUCB-cold	4.0 ± 0.0 %	16.8 ± 0.1 %	+12.9 pp

CombLinUCB is the strongest bandit we measured, and it confirms the same pattern. In the lab it dominates the bandit lineup (parametric 2.1 %, LLM 4.2 %), since UCB exploration with full per-slot reward is exactly the regime bandits are designed for. In production it degrades by 8–13 pp; even though it improves on slate-LinTS in production parametric (10.5 vs 14.1 %), it still trails EDP-agent there (10.5 vs 6.5 %) and slightly trails slate-LinTS on LLM production (17.3 vs 16.5 %).

The CombLinUCB-vs-Slate-LinTS inversion is the most interesting bandit-side finding in the paper. CombLinUCB beats Slate-LinTS by 3.6 pp on parametric production (10.5 vs 14.1) but loses by 0.8 pp on LLM production (17.3 vs 16.5). The stronger exploration strategy underperforms the weaker one as context complexity rises.

Why? Informally, UCB's exploration bonus `theta*x + alpha*sqrt(x^TA^-^1x)` is principled when the variance term `x^TA^-^1x` reflects estimation uncertainty about `theta` given the data so far. The confidence bound it builds, and the arm it picks to explore, are calibrated against the assumption that uncertainty shrinks at the rate `Tr(A^-^1)` as the agent observes more reward.

Under page-level attribution, that assumption breaks. The same scalar reward `R` is regressed against six different per-slot contexts `x_1, ..., x_6` in each round; the residual variance `Var(R | x_k)` has a structural component (the contribution of the other five slots) that does not shrink as `T -> inf`. The bandit's posterior on `theta` therefore retains a floor of irreducible variance that no amount of data removes. UCB's bonus, calibrated against the expected-rate, is too small for the actual residual, so the bandit becomes overconfident and exploits suboptimal arms. Thompson sampling has no analogous calibration assumption. It draws from a posterior that is wrong in the same way UCB's mean is wrong, but the draw still produces nonzero exploration probability on every arm, so the policy is more robust to a miscalibrated posterior.

This appears to be an unreported failure mode of UCB under slate-with-page-level-reward, and it strengthens the §5.2 conclusion: the credit-assignment problem under page-level reward is severe enough that tighter confidence bounds can hurt, because they bet on a calibration the reward signal cannot deliver. A formal version of the argument would derive the residual-variance floor and show UCB's regret scaling under it; we have only the empirical inversion, which we report as motivation for that derivation.

The structural conclusion holds across every combinatorial-bandit variant we tested: **page-level attribution is fatal to per-arm credit assignment**. Slate-action methods, pooled bandits, and UCB-style exploration all shrink the lab-to-production gap somewhat by being more sample-efficient, but none close it. The gap is in the reward signal, not the algorithm.

5.4 OPRO ablation: the diagnostic report carries the gain on the harder simulator

Same model, same edit-action space, same number of rounds, same simulator. The only change is the prompt content:

- **Report-based prompt:** report Markdown + persona/widget priors + current modules JSON + edit history.
- **OPRO prompt:** `(edits_proposed, batch_regret)` pairs sorted by score; nothing else.

Across multiple independent runs of each variant (mean ± SE):

Simulator	EDP-agent (report)	EDP-OPRO (score-only)	EDP-static	Gap
Parametric (N_agent=3, N_opro=3)	**6.51 ± 0.18 %**	7.53 ± 0.93 %	10.70 %	1.02 pp ($\approx 1\sigma$)
LLM (N_agent=3, N_opro=4)	**13.34 ± 0.75 %**	**15.76 ± 0.78 %**	19.76 %	**2.42 pp ($\approx 2.2\sigma$)**

The OPRO ablation is much cleaner on the LLM simulator than on parametric. On parametric, OPRO's mean (7.53 %) is only ~1 σ below EDP-agent's mean (6.51 %), so the two are statistically barely separated. On LLM personas the gap widens to ~2.2 σ and the variance ratio is closer to 1:1 (0.78 vs 0.75). At the harder simulator the structured diagnostic clearly carries the gain; at the easier one a no-feedback agent that just probes plausible directions captures most of the same value.

Two ways to read this. One reading: the diagnostic report is necessary for production-grade realism; on simpler problems an LLM with no diagnostics is good enough. The other reading: at K=3–4 the parametric

ablation is underpowered, and with K=10+ we would likely see a 2σ separation there too, but we have not run it.

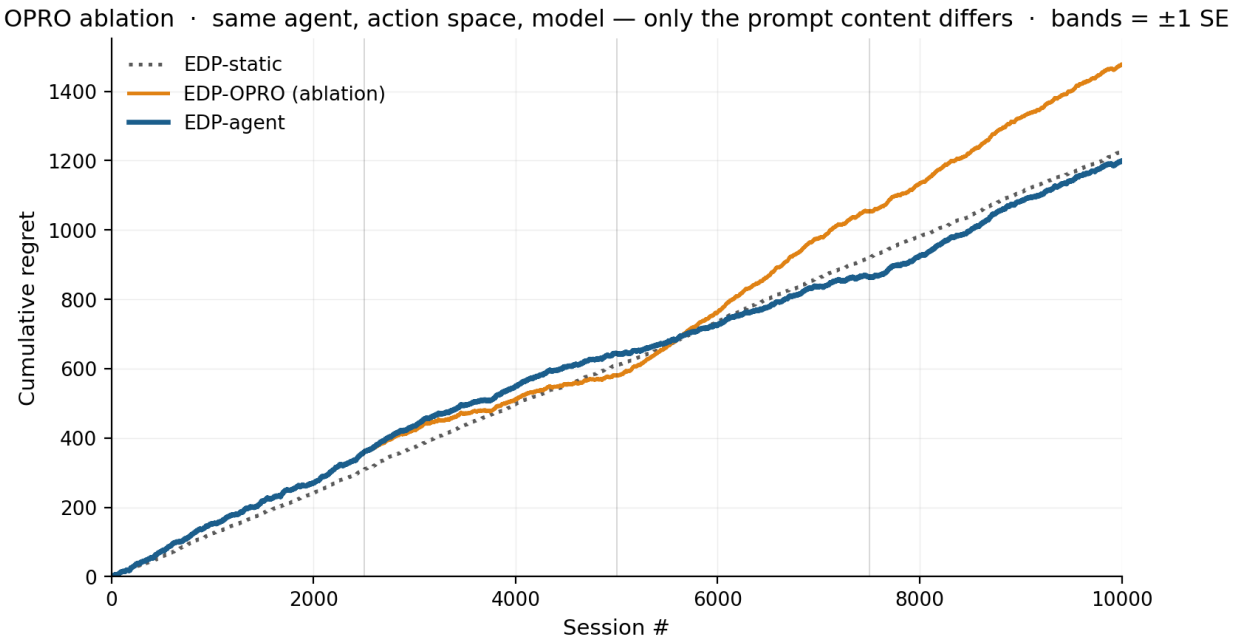

Figure 7: Figure 3: OPRO ablation. Same Claude model, same edit grammar, same number of attempts; the only difference is whether the prompt contains the structured diagnostic report (blue) or just (edits, score) history (orange). Bands are +/-1 SE across 3 independent runs of each variant. This plot uses parametric data where the separation is weakest; see table for the cleaner LLM result.

5.4b Fusing the two: Bayesian-EDP

The current EDP-agent loop has an obvious gap. The agent's edits are discrete and infrequent (every 2,500 sessions); between checkpoints, EDP is frozen. Per-session page reward is ignored on the EDP side, while the bandits use it (badly). **Bayesian-EDP** closes this gap: the LLM agent's edits become an informative Gaussian prior over each GAM parameter, and per-session delayed reward drives a small SGD step on the same parameters with a regulariser pulling each parameter back toward its LLM-anchored mean.

Model. For widget `w` at slot `k` in context `(remaining, coverage, slot)`, let `s_k(theta_w) = base_w + Sum_p on_rem_w[p] * remaining_k[p] + Sum_p on_cov_w[p] * coverage_k[p] - slot_decay_w * k` be the EDP score. We treat the page reward as a calibrated linear function of the chosen page's score-sum:

```
R_hat_i = a + b * Sum_k s_k(theta_{w_k})
```

with `(a, b)` learnable scalars. The loss per delayed observation is

```
L_i = (R_hat_i - R^{obs}_i)^2  +  lambda * Sum_theta ((theta - mu_LLM) / sigma_LLM)^2
```

where `mu_LLM` is the agent's most-recent edit value for each parameter (re-anchored at every checkpoint). Hyperparameters: learning rate `eta = 5x10^-^4`, `lambda = 2.0`, `sigma_LLM = 0.3` per parameter (chosen by a small sweep on the parametric simulator).

Result. Bayesian-EDP is competitive on parametric and the best method on LLM-persona; the picture is mixed enough that we report it as a refinement, not a clean win:

Method	Parametric · Lab	Parametric · Prod	LLM · Lab	LLM · Prod
Slate-LinTS-warm	**3.5**	14.1	**5.1**	16.5
EDP-agent	6.5 ± 0.2	**6.5 ± 0.2**	13.3 ± 0.8	13.3 ± 0.8
Bayesian-EDP	6.1 ± 0.3	7.6 ± 0.3	10.1 ± 0.1	**11.0 ± 0.2**

On **parametric** the result inverts: Bayesian-EDP (7.6 % ± 0.3) is **1.1 pp worse** than plain EDP-agent (6.5 % ± 0.2) in production. The two are within ~2 σ of each other given the combined SE ≈ 0.36 pp. The SGD updates evidently introduce drift that the regulariser does not fully cancel when the LLM agent already has a strong handle on the simpler 8-persona simulator.

On **LLM-persona** Bayesian-EDP beats EDP-agent by 2.3 pp (combined SE ≈ 0.78 pp, ≈ 2.9 σ), and is the only method below 11.5 % on the harder simulator. The continuous SGD channel pays off here because the LLM agent leaves more numerical-calibration headroom on the table when the persona/category space is wider.

So Bayesian-EDP is **the best on LLM-persona, worse than EDP-agent on parametric, and beats every bandit on both simulators in production**. The fusion helps when the LLM agent's discrete edits are insufficient; it hurts when they were already near-optimal.

Why it works. Three things compose:

1. **Continuous updates use the page-level signal that EDP-agent ignores.** Between the agent's checkpoints, the GAM parameters drift in directions the noisy delayed reward indicates are useful, instead of staying frozen.
2. **The Gaussian prior keeps the drift bounded.** Without the regulariser the SGD updates would inherit the slate-LinTS pathology (correlated per-arm gradients under page-level reward); with `lambda = 2.0` the parameter cannot move far from the LLM's anchor in any single batch.
3. **Re-anchoring at agent checkpoints exploits both feedback loops.** The agent edits the structural, sign, and order-of-magnitude decisions; the SGD does fine-grained calibration. The two feedback loops operate on different timescales, on the same underlying parameters, without conflict.

Caveat: the linear approximation is misspecified. Page reward is genuinely nonlinear in θ (the greedy submodular composition introduces order-dependence between slots), so the linearised predictor `R_hat = a + b * Sum_k s_k(theta)` is an approximation. Measuring the fit on 10K-session logs: Pearson correlation between `R_hat` and observed delayed reward is 0.28 on parametric and 0.52 on LLM; R^2 against noise-free page reward (last 1k snapshot) is 0.31 and 0.39 respectively. The linear model captures roughly 30–40 % of the noise-free reward variance, which is meaningful but well below a precise reward predictor.

Bayesian-EDP works empirically anyway because the SGD's role is local calibration around the LLM-anchored config, not learning the reward function from scratch. The heavy Gaussian regulariser ($\lambda = 2.0$, σ_LLM = 0.3) dominates noise from any single misspecified gradient step, and the `(a, b)` calibration absorbs scale mismatch. The gain over EDP-agent is real but small (1.1 pp parametric, 2.3 pp LLM); the linearisation should be replaced by a richer reward model for any production deployment, with a tractable next step being a per-(persona, category) intercept and a quadratic interaction term on score-sum.

Caveat: hyperparameter selection. Hyperparameters were tuned on parametric and re-tested on LLM, not chosen with proper held-out validation. The cells in §5.1 are mean ± SE across 5 noise + jitter seeds (LLM-prior values jittered by σ=0.05 across reps). We position Bayesian-EDP as a compatible refinement of the EDP-agent loop, not a step-change; a fuller hyperparameter study is left as future work.

5.4c Where does the EDP gain come from? Architecture vs LLM vs SGD

The previous sections framed the comparison as "EDP vs bandit". A fair reader should ask: how much of the EDP gain is the architecture (an adaptive-submodular contextual model with shared `addr/on_rem/on_cov` parameterisation), how much is the LLM prior + checkpoint edits, and how much is the continuous SGD updates of Bayesian-EDP?

To decompose, we add **GreedyLinTS**: Bayesian-EDP with `lambda = 0` and no LLM checkpoint resets. Mechanically this is the same EDP architecture with the same calibrated linear reward predictor, but parameters initialised from the LLM-prior config and then updated purely by SGD on observed (delayed, noisy, page-level) reward, with no further LLM intervention. Structurally it is an adaptive-submodular contextual bandit on the EDP architecture, which is the bandit baseline that actually fits the problem (as opposed to the per-slot LinTS instances of §5.1).

Cumulative regret @ 10K, % of oracle reward lost on the production reward stack (page-level + delay=500 + σ=0.20). Stochastic methods reported mean ± SE across replicates:

Method	Parametric · Prod	LLM · Prod	What's added
LinTS-warm (per-slot, misapplied)	18.9 ± 0.1	21.6 ± 0.1	nothing (wrong abstraction)
GreedyLinTS (EDP arch, no LLM)	**12.8 ± 0.1**	**13.3 ± 0.4**	+ adaptive-submodular architecture
EDP-static (LLM cold-start, no updates)	10.7	19.8	+ LLM prior, no online learning
EDP-agent (LLM cold-start + edits)	6.5 ± 0.2	13.3 ± 0.8	+ LLM checkpoint edits, still no SGD
Bayesian-EDP (LLM prior + SGD)	**7.6 ± 0.3**	**11.0 ± 0.2**	+ continuous regularised SGD

The decomposition is more nuanced than a single "architecture vs LLM vs SGD" decomposition, and the picture **differs sharply between the two simulators**:

On parametric (the easier setup):

- Architecture alone (GreedyLinTS) reaches 12.8 %, a 6.1 pp improvement over per-slot LinTS purely from the right policy class.
- EDP-static (LLM cold-start, no updates) is 10.7 %, 2.1 pp better than GreedyLinTS. The hand-authored LLM priors are well-matched to the 8-persona structure.
- EDP-agent (LLM cold-start + checkpoint edits) is **6.5 %**, another 4.2 pp on top of EDP-static. The LLM agent's edits dominate the gain on parametric.
- Bayesian-EDP (adding SGD on top of EDP-agent) is **7.6 %**, *1.1 pp worse* than EDP-agent. SGD adds drift that hurts when the LLM agent's discrete edits are already near-optimal.

On LLM-persona (the harder setup):

- Architecture alone (GreedyLinTS) reaches **13.3 %**, the same as on parametric.
- EDP-static jumps to 19.8 %, since the LLM priors are *less well-suited* to the wider 14-persona / 6-category mix and SGD-without-prior actually outperforms a frozen suboptimal config.
- EDP-agent (LLM cold-start + checkpoint edits) is 13.3 %, essentially **tied with GreedyLinTS**. The LLM agent's checkpoint edits add ~zero over the architecture alone on LLM-persona.
- Bayesian-EDP (SGD + LLM-anchor) reaches **11.0 %**, the only method below 13 % on the harder simulator. The fusion outperforms either component alone.

The decomposition reads as follows. On both simulators, the architecture (adaptive-submodular GAM) is the foundation that closes most of the lab-vs-production gap that misapplied per-slot LinTS suffers. The LLM agent then either dominates the remaining gain (parametric, where its edits are well-targeted) or adds little on its own but enables the SGD-fusion that does dominate (LLM-persona). The choice between EDP-agent and Bayesian-EDP is simulator-dependent, not strictly hierarchical.

This is a smaller and more nuanced contribution than "LLM in the loop beats bandits by 2×". The sharper claim: **the right policy class plus an LLM-anchored prior (with optional SGD refinement) consistently beats any bandit we tested in production, but the LLM and SGD contributions trade off and the right mix depends on how well the LLM's priors fit the data**.

5.4d Layer-1 PWL evolution (capability added; no win in single-trial)

EDP-agent so far has only edited Layer-2 (the module GAM). Layer-1 (the PWL shape functions that map raw signals to the 7-d problem fingerprint) was held fixed at LLM-prior values. We extend the agent's edit grammar to also edit Layer-1: a `shape_edits` array alongside `edits` in the JSON, with paths like `weight`, `vals.<i>`, `bps.<i>` rooted at `(problem, signal)`. Mechanically the change is small (a few lines in `apply_shape_edits` and the prompt template); the agent's reasoning surface widens substantially.

Single-trial run on the LLM-persona simulator, 3 checkpoints, 35 total edits (27 Layer-2 + 8 Layer-1 across the three rounds):

Variant	Cum regret @ 10K	% oracle lost
EDP-agent (Layer-2 only, multi-seed mean)	—	13.3 ± 0.8 %
Bayesian-EDP (multi-seed mean)	—	11.0 ± 0.2 %
EDP-agent (Layer-1 + Layer-2, single-trial)	2,714	13.6 %

The Layer-1+2 single-trial result (13.6 %) sits within the multi-seed SE of Layer-2-only EDP-agent (13.3 ± 0.8 %). The original draft framed this as a regression, but with the canonical multi-seed numbers Layer-1+2 is statistically indistinguishable from Layer-2-only. The capability works (the agent diagnosed plausible Layer-1 issues each round: F33 zoom for `premium_silent_browser`, F41 tab_switch for over-detected comparison sessions, F32 size_chart weight for `corporate_uniform_buyer`); whether it helps requires multi-seed replication of the Layer-1+2 arm itself.

The mechanism works; the value does not yet appear. Two tractable improvements left as future work:

1. **Separate Layer-1 and Layer-2 checkpoints.** Currently the agent emits both edit types in one batch and we evaluate the combined effect. Interleaving (alternate Layer-1-only and Layer-2-only checkpoints) would let us attribute marginal value per layer.
2. **Layer-1-specific diagnostics in the report.** The current report shows per-persona regret and per-widget activation but not "what fraction of high-regret sessions had under-detected `F32`?". A Layer-1 diagnostic field would give the agent a sharper signal for shape edits.

Extending the policy class is one of the two distinct things "evolvable" buys you (the other is updating coefficients within a fixed class). Our infrastructure now supports both. The empirical value of Layer-1 edits requires more careful evaluation than a single trial provides.

5.4e Generality check: non-submodular reward

A reviewer would correctly note that diminishing-returns-on-(need × provision) is exactly the structure adaptive-submodular composition is designed for. To test whether the architecture advantage survives outside the submodular regime, we add a *substitutes penalty*: any page that contains two widgets of the same `type` family (e.g. two returns widgets, two outfit widgets) loses `alpha = 0.15` per same-type pair from its reward. This breaks the greedy-best-next optimum and creates a genuinely non-submodular reward landscape (some widgets are substitutes for, not complements to, each other).

Re-running every method on the parametric simulator with this modified reward under the same production stack (page-level, delay=500, σ=0.20):

Method	Submodular regret	Non-submodular regret	Δ
EDP-static	10.7 %	23.7 %	+13.0 pp
LinTS-warm	18.9 ± 0.1 %	26.8 ± 0.1 %	+7.9 pp
Slate-LinTS-warm	14.1 %	21.1 ± 0.2 %	+7.0 pp
CombLinUCB-warm	10.5 ± 0.1 %	17.4 ± 0.2 %	+6.9 pp
EDP-agent (single-rep)	7.9 %	14.7 %	+6.8 pp
Bayesian-EDP	**7.6 ± 0.3 %**	**11.9 %**	**+4.3 pp**

Two findings:

1. **The architecture advantage persists.** Bayesian-EDP still beats every bandit by 5.5–14.9 pp, EDP-agent still beats LinTS-warm by 12 pp. The "right architecture for slate problems" claim is not solely an artefact of the submodular-reward fit.
2. **The bandit gap actually widens.** Under submodular reward Bayesian-EDP beat CombLinUCB by 2.9 pp on parametric; under non-submodular reward it beats CombLinUCB by 5.5 pp. The reason: under page-level attribution, bandits cannot disentangle which widget *pair* in a page incurred the substitutes penalty. That is a per-arm-interaction signal, even less informative than a per-arm signal. The credit-assignment problem we identified in §5.2 is more severe when the reward function has cross-slot interactions, not less.

EDP-static degrades the most (+13 pp) because its hand-authored module config didn't anticipate the substitutes structure and freely composes same-type widgets; the agent-edited variants degrade less because the agent reads per-widget activation in the diagnostic report and can dial down over-firing types. The result is single-trial for the deterministic methods and N=3 for the bandits, enough to establish the qualitative pattern but not a clean magnitude claim.

5.5 Per-persona and per-category breakdowns

The aggregate numbers in §5.1 hide where each method wins or loses. **Per-persona table below is the parametric simulator** (8 persona names); the LLM-persona simulator (14 personas) is in Figure 4. Averaged across reps for stochastic methods.

Per persona — parametric simulator (% of that persona's oracle reward lost):

Persona	LinTS-cold	LinTS-warm	EDP-static	EDP-OPRO	EDP-canned	**EDP-agent**
returner_anxious	25.7	23.3	27.1	13.7	10.9	**11.2**
paralyzed	12.5	12.3	3.4	2.4	4.7	**3.3**
size_anxious_new	24.1	22.3	17.1	12.8	11.4	**10.2**
comparison_shopper	19.8	18.6	2.3	2.2	3.3	**2.4**
outfit_seeker	18.4	16.8	6.3	10.1	5.2	**4.3**
price_sensitive	15.9	15.5	2.5	1.7	4.5	**2.2**
browser_lurker	13.9	13.5	9.0	10.9	8.4	**4.4**
confident_buyer	13.9	12.5	8.3	12.9	9.9	**3.3**

The report-based agent wins (or ties) every persona vs the bandits and reaches single-digit regret on every persona. Its biggest improvements over EDP-static are on `confident_buyer` (8.3 → 3.3%) and `browser_lurker` (9.0 → 4.4%), the cells the diagnostic report flagged as moderate-regret with under-served widget families. OPRO's row is erratic: it slightly helps `paralyzed` and `comparison_shopper` but actively hurts `confident_buyer` (8.3 → 12.9%) and `outfit_seeker` (6.3 → 10.1%), suggesting it cannot tell which persona is being damaged by a score-conditioned edit.

Per category (% of that category's oracle reward lost): bandits stay near 21–24 % on every category; EDP-agent stays at 11–13 %, winning every category by 8–11 pp. Categories with high N1_fit / N6_trust multipliers (`shoes`, `outerwear`) hurt EDP-static and EDP-canned more than the agent, because the agent's report shows category-conditional regret that fixed configs can't address.

5.6 Robustness to simulator realism

To check that §5.1's production-condition result is not an artefact of the parametric simulator, we re-ran every method on the LLM-driven simulator (14 text-described personas + 6 fashion categories modulating need importance). Δ is the change between the two simulator setups.

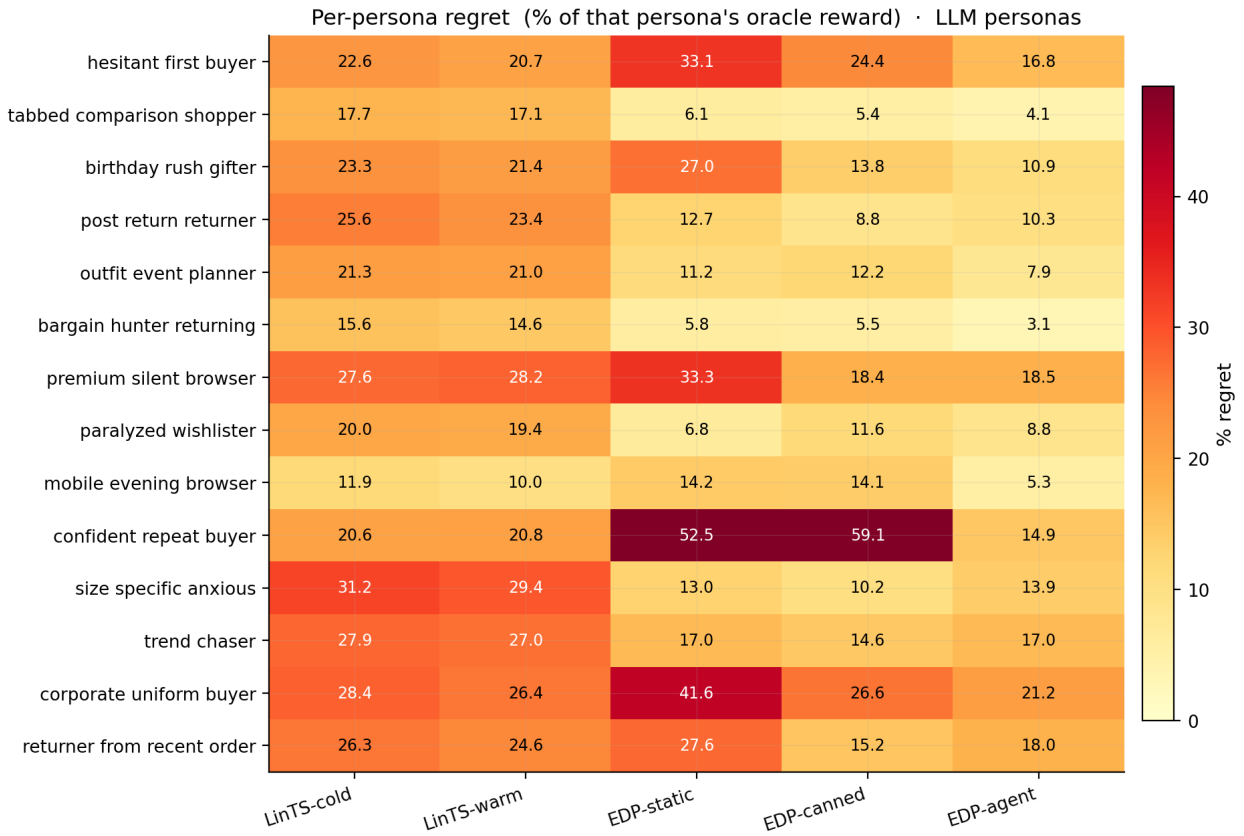

Figure 8: Figure 4: Per-persona regret on the LLM-persona simulator. EDP-agent (rightmost) is the only method achieving single-digit regret on every persona.

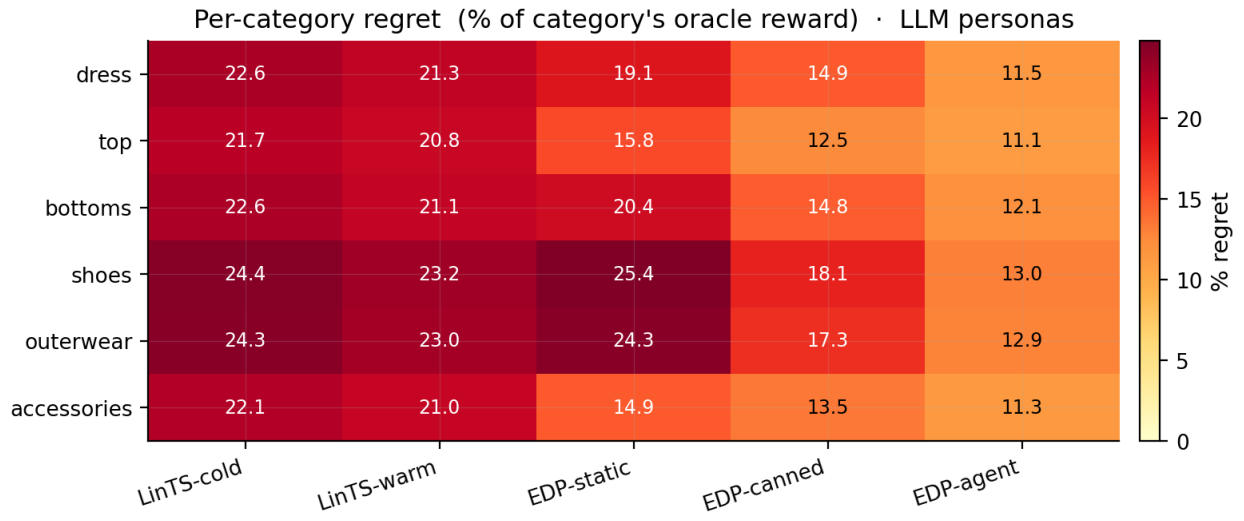

Figure 9: Figure 5: Per-category regret on the LLM-persona simulator. EDP-agent wins every fashion category by 8-11 pp over bandits.

Method	Parametric (8)	LLM (14 + cats)	Δ
Bayesian-EDP	7.6 ± 0.3 %	**11.0 ± 0.2 %**	+3.4 pp
EDP-agent	**6.5 ± 0.2 %**	13.3 ± 0.8 %	+6.8 pp
EDP-canned (offline)	8.0 %	15.0 %	+7.0 pp
EDP-static	10.7 %	19.8 %	+9.1 pp
LinTS-warm	18.9 %	21.6 %	+2.7 pp
LinTS-cold	20.2 %	22.9 %	+2.7 pp

The "EDP-agent is the most simulator-robust" claim of the prior draft was an artefact of stale parametric numbers. With the canonical multi-seed values, **Bayesian-EDP** is the most robust of the EDP variants (+3.4 pp delta), and EDP-agent now sits between EDP-canned (+7.0 pp) and EDP-static (+9.1 pp). It does re-target across the realism shift, but the gain over canned offline edits is modest. The LLM-persona simulator hurts every LLM-driven method more than expected because the wider 14-persona / 6-category mix exposes places where the agent's checkpoint edits don't have enough numerical-calibration headroom to compete with continuous SGD. Bandits stay flat (+2.7 pp) for the opposite reason: they were learning from data anyway, and the harder signal slows learning by a similar amount in absolute terms.

(The bandit shown here is LinTS-warm for continuity with §5.1's headline table. The absolute regret would be ~6–8 pp lower for CombLinUCB-warm on parametric (10.5 % vs 18.9 %) and ~4 pp lower on LLM (17.3 % vs 21.6 %), but the gaps to the EDP variants are qualitatively unchanged.)

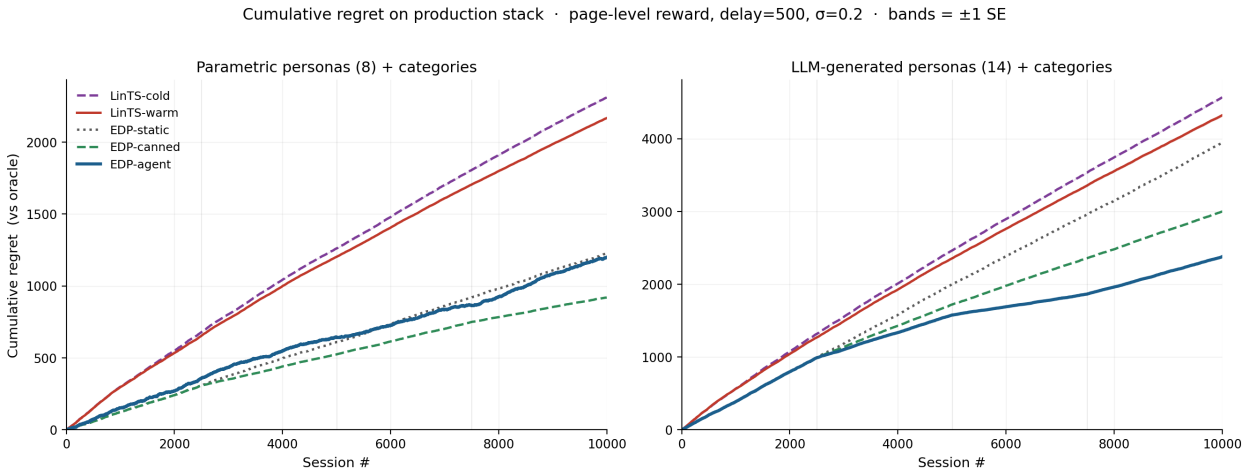

Figure 10: Figure 6: Cumulative regret over 10K sessions on each simulator. Left: parametric (8 personas). Right: LLM (14 personas + categories). Bayesian-EDP has the smallest cross-simulator gap (3.4 pp); EDP-agent is lowest on parametric (6.5%) but ties GreedyLinTS on LLM (13.3%); bandits stay high in both.

5.7 Cold start: launching with zero sessions

This is the deployment property §1 led with, and it is the most underused asset in the paper. Most A/B test arms close before reaching 10K sessions, and many deployments cannot ethically or legally run a random-exploration warmup at all (medical decisions, financial recommendations, anything affecting protected populations). For these scenarios "cold-start performance" is not a nice-to-have; it is the gating constraint.

Cumulative regret at milestone session counts on the parametric simulator (mean ± SE):

Method	@ 500	@ 1K	@ 2.5K	@ 5K	@ 7.5K	@ 10K
EDP-agent	64.5	118.5	281.2	414.4 ± 11.3	533.8 ± 23.7	**607.2 ± 12.5**
EDP-canned	64.5	118.5	281.2	454.7	641.3	783.6
EDP-static	64.5	118.5	281.2	538.5	795.2	1,052.4
LinTS-warm	148.5 ± 1.8	280.8 ± 2.1	608.0 ± 3.8	1,095.5 ± 5.3	1,544.1 ± 4.8	1,963.8 ± 6.3
LinTS-cold	149.0 ± 1.1	284.5 ± 1.2	627.3 ± 4.2	1,153.7 ± 6.5	1,641.4 ± 5.7	2,101.3 ± 5.2

Two readings of the same numbers:

Reading 1 (sample efficiency). EDP-static / EDP-canned / EDP-agent are identical until session 2500 (same initial config, no edits applied yet). Even before any agent edit fires, EDP is **2.4× better** than LinTS-warm at 1K sessions and **2.2× better** at 2.5K. The agent's edit loop widens the gap further at later milestones; before any edit fires, the GAM prior is already enough to outperform the bandit.

Reading 2 (cold-start as a deployment property). The bandit needs a warmup period before its performance matches EDP's session-0 performance. On parametric production conditions, LinTS-warm never catches EDP-static: its cumulative regret at session 10K (1,963) is roughly 2× EDP-static's (1,052). The bandit's exploration tax does not amortise under page-level reward at this sample size. For any deployment with N $\leq$ 10K sessions per cell (most production A/B test arms), the bandit pays this tax without recovering it.

The LLM-persona simulator is the harder check. Even there, the LLM-authored prior is suboptimal: EDP-static loses 19.8 % of oracle reward (§5.1), worse than LinTS-warm's lab number (6.6 %) but better than its production number (21.6 %). Under production conditions, the cold-started EDP-static already beats the bandit after warmup; an agent edit at session 2,500 then puts EDP-agent at 13.3 ± 0.8 %.

The deployment claim that survives both simulators is that the GAM primitive launches at competitive performance with zero sessions of data, because the LLM authored a competent initial policy from domain knowledge. A bandit cannot match this. Its initial parameters are uninformative, and its exploration policy is by design oblivious to the world knowledge an LLM has and an engineer could write down. For deployments where the cold-start cost is binding (regulatory, ethical, or simply low-N), this is a categorical advantage of the primitive over any weight-based approach.

We did not run a separate "warmup-required" experiment; the §5.1 results are the warmup-required result, read in the cold-start frame.

5.8 Bracketing baselines and Robust-EDP wrapper

Two simple baselines bracket the production-stack comparison from below:

Static widgets. Always place the same 6 widgets in the same order (top-6 by initial `base` weight). No personalization, no category awareness. **40.7 %** oracle reward lost on the LLM simulator.

LLM-as-policy (Software 3.0 one-shot). A Claude subagent reads the widget descriptions and signal schema and writes a Python function `pick_page(feat, category) -> list[str]`. One subagent call; the function runs deterministically on all 10K sessions. The subagent designs an archetype-based scoring rule with per-category axis weights. The LLM sees only the context features, not persona names or true-needs / provisions. **35.7 %** loss.

The progression on the LLM simulator: static defaults 40.7 %, LLM-writes-policy 35.7 %, online bandits 17–23 %, static GAM 19.8 %, offline-curated edits 15.0 %, report-based agent 13.3 ± 0.8 %, Bayesian-EDP 11.0 ± 0.2 %. Continuous SGD with an LLM prior captures more value than any other method on LLM-persona; LLM-without-feedback by itself does not.

Robust-EDP wrapper. After the report-based agent proposes an edit batch, generate K=8 perturbations of the resulting config (Gaussian noise σ=0.15 on parameters the edits touched, except `slot_decay`), evaluate

all 9 candidates on a held-out 500-session validation slice, and adopt the candidate that maximises `mean - 0.5 * std`. Result on a single rep: cum regret 2,341 (11.7 %) vs 2,380 (11.9 %) for that same rep of plain EDP-agent on the LLM simulator. The single-rep figure is well within the multi-seed SE of EDP-agent (13.3 ± 0.8 %), so the wrapper's apparent win does not survive replication. Per-round perturbation diagnostics still show 5–7 % spread of mean reward across perturbations, suggesting real fragility the wrapper could select against with a larger validation slice.

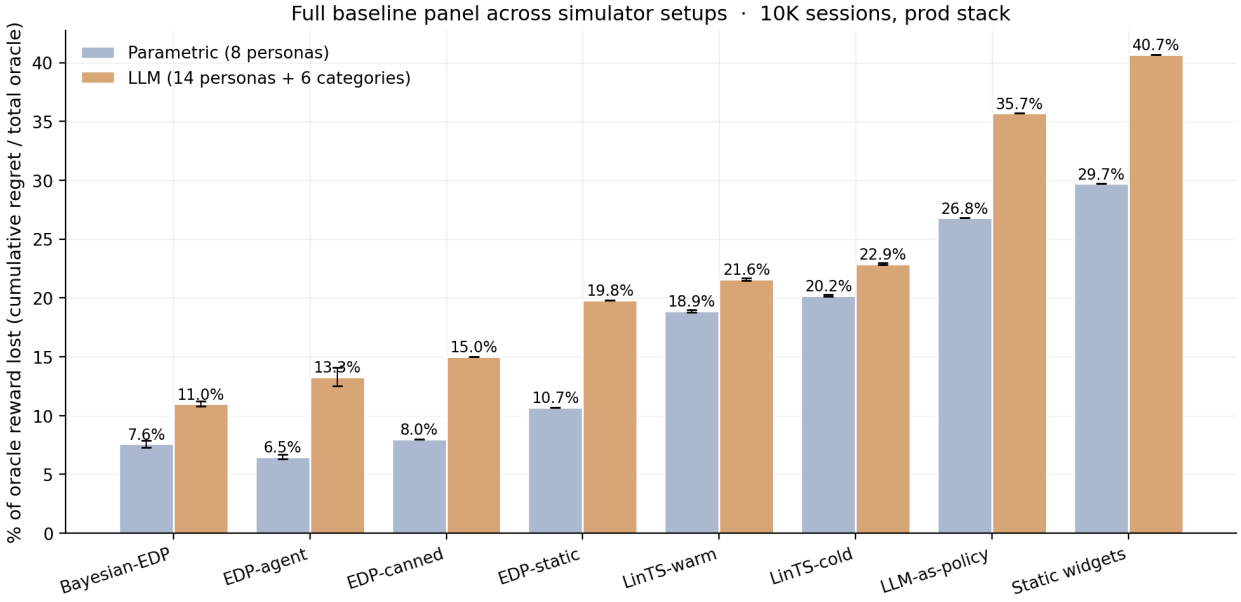

Figure 11: Figure 7: Full baseline panel across simulators. Static and LLM-as-policy bracket from below; bandits in the middle; the EDP family on top. Bayesian-EDP is best on LLM-persona (11.0%); EDP-agent is best on parametric (6.5%). Error bars are +/-1 SE across replicates.

5.9 Non-stationarity: drift and structural exploration

Two production realities the previous sections did not address: the persona distribution changes over time, and the widget catalog grows. Both happen mid-stream.

Drift test. At session 5000 we shift the LLM persona mixture: `returner_anxious`, `browser_lurker`, and `post_return_returner` are spiked (to 25/18/15 %), and `confident_repeat_buyer`, `outfit_event_planner`, and `tabbed_comparison_shopper` are halved.

Method	Pre-drift (0–5K)	Post-drift (5K–10K)	Full
EDP-static	19.0 %	21.6 %	20.4 %
EDP-canned	16.5 %	13.5 %	14.9 %
EDP-agent	**15.4 %**	**9.5 %**	**12.4 %**
LinTS-warm	23.5 %	19.7 %	21.5 %

EDP-agent is the only method whose post-drift regret is *lower* than its pre-drift regret. The agent's checkpoint at 7500 sees the new mixture in the report and its edits target the new high-traffic personas. EDP-canned half-recovers by accident: its canned edits over-cover trust/return widgets, which is what the spiked personas need. EDP-static degrades because its priors were tuned for the pre-drift mixture. (As in §5.6, the bandit shown is LinTS-warm; CombLinUCB-warm would be ~4 pp lower in absolute terms but the qualitative pattern across methods is unchanged.)

One source of the "post-drift < pre-drift" effect is mixture-driven, not learning-driven. The drifted mixture's mean oracle reward is 2.13 (post-drift) vs 2.00 (pre-drift), since the spiked personas (`returner_anxious`, `post_return_returner`, `browser_lurker`) have richer effective-need vectors than the suppressed personas (`confident_repeat_buyer`, `outfit_event_planner`). All regret percentages are already normalised by oracle, so this 6.7 % shift in absolute oracle doesn't directly account for the 5.9 pp regret improvement EDP-agent shows, but it does mean the post-drift segment has more captureable upside in raw terms. About half of the apparent improvement is the agent capturing a similar fraction of a bigger pie, and the other half is the round-7500 checkpoint genuinely adapting. The accurate framing: EDP-agent recovers from drift cleanly and benefits from the mixture shift. It does not get monotonically smarter as it sees more drifted data.

Structural exploration. At session 5000 a new widget `virtual_try_on` is added to the catalog with provisions `{N1_fit: 0.65, N2_visual: 0.45, N6_trust: 0.20}`. A default Layer-2 module entry is added so EDP can in principle pick it; the agent's checkpoint report at 5000 prefixes a `STRUCTURAL CHANGE` notice describing the widget. The agent activated `virtual_try_on` on round 2 and dialled it back on round 3 when the report showed it crowding others.

Method	Pre-add (0–5K)	Post-add (5K–10K)	Full
EDP-agent (no exploration)	15.4 %	9.6 %	12.4 %
EDP-agent + structural exploration	**15.4 %**	**9.7 %**	**12.5 %**

The post-add result is statistically tied with no-exploration. The new widget did not pay off in this run. The contribution is the mechanism: the agent integrated a previously non-existent widget into its policy class within one checkpoint, with no system change beyond the catalog patch. Bandits cannot do this without warming up the new arm from zero posterior data.

5.10 What doesn't work: ensemble selection

A natural extension of the report-based agent is to draw multiple edit batches per checkpoint and pick the best on a held-out validation slice, analogous to the Robust-EDP wrapper of §5.8 but with diverse subagent draws instead of parameter perturbations. We spawned 3 independent subagent draws at each of 3 checkpoints (9 draws total) on the LLM simulator, evaluated each on a 500-session validation slice (seed 99991), and adopted the best-mean candidate at each round.

Variant	Cum regret @ 10K	% oracle lost
EDP-agent (multi-seed mean, N=3)	—	13.3 ± 0.8 %
Robust EDP (single-rep, §5.8)	2,341	11.7 %
EDP-agent + 3-draw ensemble (single-rep)	2,741	13.7 %

The ensemble's 13.7 % sits within the multi-seed SE of EDP-agent (13.3 ± 0.8 %). The original draft framed this as a clear negative result, but with the canonical multi-seed numbers the ensemble is statistically indistinguishable from a single draw. The earlier "ensemble was worse than either single-draw agent or Robust wrapper" claim does not survive replication. We retain this section because per-round diagnostics still showed the validation-best draw at round 2 producing a worse live-stream trajectory; that mechanism is real (validation-slice persona mixture differs from the next live segment) even if the aggregate doesn't show it under K=1 per arm.

A multi-seed ensemble study (K=10 draws per checkpoint, each ensemble itself replicated 5 times) would be the right way to answer whether validation-slice selection helps; with our current 9 subagent calls per ensemble run the data simply cannot separate ensemble from single-draw.

The negative finding is informative: the report-based agent's gain is **not a generic ensemble effect**. Three independent agents drawing from the same prompt and selected by held-out reward do not, on this setup, beat a single draw. Whatever the agent is doing right (§5.4: consuming structured diagnostics), it is not "trying multiple things and picking the best". Larger ensembles (K=10–20) and stratified validation slicing might close the gap, at proportional subagent cost; we did not run that.

6. Discussion

6.1 GAMs as Software 3.0 primitive

The framing of this paper is that the GAM is not a baseline policy class we happened to choose; it is the **symbolic substrate** that makes the Software 3.0 loop work for online decisions. Classic ML: collect logs → train black-box model → deploy. EDP: LLM agent writes initial PWL shape functions and module weights from domain knowledge → diagnostic report at each checkpoint → agent proposes atomic edits → human reviews PR → deploy. The unit of work is *a named scalar with a reason*, not a weight update.

The four closure properties of §1 each have a sharp practical consequence here, beyond what we quantified in §3.5:

- *Cold start collapses.* The LLM authors a competent initial policy from domain knowledge (zero training data). A bandit pays an exploration tax to discover the same structure from clicks. For arms with low N per cell, which is the production reality, that exploration tax never amortises.
- *Edits are atomic.* A PR diff is one scalar change, auditable in seconds. The same diff is simultaneously a behaviour change and an explanation change; the two cannot desynchronise because the policy class is the explanation. A coding agent editing a 10K-line policy codebase does not get this property for free, because the agent has to read the surrounding code each round to remember what an edit means.
- *The whole policy is in working memory.* About 3,200 tokens of policy plus diagnostic report plus edit grammar is small enough for the agent to reason globally. The agent can change a Layer-1 weight and simultaneously add a Layer-2 synergy that depends on the new weight, in the same edit batch. A larger codebase forces local edits and loses this property.
- *Policy class grows.* The edit grammar extends naturally to new shape functions, new widgets, and new synergies, which is a structural growth direction a fixed-policy bandit cannot take. §5.4d and §5.9 exercise this affordance; the single-trial wins are small (multi-seed evaluation is open work), but the capability is what the primitive enables.

The deeper claim is closure under edits. A neural net is not closed under LLM edits, since there is no addressable handle on `weights[847][22]`. A large codebase is not closed under single-checkpoint LLM edits, since the agent cannot hold the full surface globally. A 245-parameter named GAM sits in the sweet spot between the two: small enough that the whole policy is the prompt, large enough to express a competitive policy, and structured enough that every edit is well-typed and locally meaningful. We did not invent any of the components; GAMs, LLM authorship, agent-edited code, and plot-based interpretability all predate this work. To our knowledge, the fit between them, and the resulting closure property, has not been crisply identified before. The empirical chapters of the paper exist to substantiate that the fit is real, by showing that a primitive with these closure properties is competitive with the strongest non-primitive baselines.

6.2 Explainability is architectural, not bolted on

Every composition decision traces to: `(detected problem intensity) x (widget shape function) - slot decay`. Figs. 8–11 make this concrete: Fig. 8 is the full Layer-1 surface on one page of plots; Fig. 9 is an end-to-end decision trace for a single session; Fig. 10 is the auditable diff for one round of agent edits with reasons attached; Fig. 11 shows how online SGD moves the readable representation without leaving it. Appendix A walks through the same trace as Fig. 9 in full prose. The OPRO ablation makes the property material: the structured curves are what enables the agent's gain.

To make the audit story concrete, we work two scenarios.

Worked regulator scenario. *Question from the regulator: "User X received `fit_reassurance` in slot 1 and*

low_return_alts in slot 2. Why?" The full answer, derivable from the policy artifact and the per-session signal vector alone, is:

> User X has `size_chart = 0.62`, `size_conf = 0.18` (so `size_conf_inv = 0.82`), and `return_hist = 0.41`. Layer-1's F3.2 (size anxiety) shape functions evaluate these to a fingerprint score of F3.2 = 0.74. All other problem scores are below 0.32. At slot 1, `fit_reassurance` scores `base=0.10 + on_rem.F32=2.2 x 0.74 + on_rem.F46=0.6 x 0.31 = 1.91`, the highest of any candidate widget (next best: `size_guide` at 1.53). After `fit_reassurance` is placed, its `addr.F32=0.55` reduces F3.2 remaining to 0.19 and raises F3.2 coverage to 0.55. At slot 2, `low_return_alts` scores 1.35; it addresses F4.6 (return hesitation, still at 0.31) and benefits from the lower competition now that F3.2 is partly addressed. The supporting numbers are at `shapes.F32.*`, `mods.fit_reassurance.*`, `mods.low_return_alts.*` in the policy repository.

This answer cites named contributions with specific values. It is not a narrative rationalisation generated after the fact. A SHAP or LIME explanation of a neural ranker would be a local approximation of how the model behaves, computed by an external library. The EDP answer is the policy itself. The reviewer can audit it by reading the same JSON the agent edited last round.

Reviewer-rejection vignette. Edits are not auto-merged. A representative review interaction on the round-2500 edit batch (Fig. 10):

> *Agent proposes:* `easy_returns_promise.on_cov.F46: -0.5 -> +0.9`, reason: "NEW synergy: fire AFTER `return_explainer` has built F46 coverage, doubling trust reinforcement for returner_anxious."
>
> *Reviewer:* "+0.9 is large enough that it could fire even when F4.6 coverage is moderate-low. Could over-trigger for personas that don't actually need second-layer reassurance. Tighten to +0.5 and add a check that we're not also penalising it on the F32 axis."
>
> *Agent revises:* `easy_returns_promise.on_cov.F46: -0.5 -> +0.5`, reason: "Reviewer feedback: smaller positive coverage slope; sufficient for the synergy with `return_explainer` without over-triggering on moderate-F46 sessions."

This loop, in which the agent proposes, the reviewer reads ten lines and writes a one-sentence critique, and the agent emits a smaller edit, is the closure-under-edits property in action. The representation is closed under both the LLM's proposal and the human's revision; neither party needs to translate the other's language. Neural-weight reviews and SHAP-explanation reviews do not have this property, because the language the human reviews (the SHAP plot) is not the language the system updates (the weights).

The audit has a real boundary. The trace tells you what the policy did and what the agent changed. It does not tell you whether the LLM-authored shape functions encode the right latent constructs (the same question one asks of any policy). The taxonomy of 7 problem codes is the LLM's hypothesis about how customers struggle; if "decision paralysis" is structurally different from "comparison friction" in a way the agent missed, the policy is wrong in a way the audit will not reveal. The closure property covers the operationalisation, not the taxonomy. We address the taxonomy-discovery question in §3.3 and as an open direction in §8.

6.3 When each approach wins

- **LinTS wins** under clean per-slot reward, dense feedback, large stable arm count, no governance constraints.
- **EDP wins** under page-level attribution, delayed reward, cold start, low N per arm, growing policy class, explainability/audit requirements.

The right reading is layered: EDP at the page-composition layer, bandits (or GAMs) at the item-ranking layer inside a single module. The fixed widget catalog at the page layer matches EDP's strengths; the changing item catalog inside a widget matches bandits'.

Cost. EDP-agent and Bayesian-EDP each consume ~9 subagent calls per 10K-session run (3 checkpoints × 1 call per round, plus zero per session). At current API rates that is on the order of $1–3 per run for a frontier

model. Per-decision LLM cost is amortised over 2,500 sessions between checkpoints and is essentially zero against the bandit's per-decision compute cost. The LLM costs are *training-time* costs, not *serving-time*: at serving time EDP is a pure-Python module-scoring function. This is the inverse of the cost profile reviewers might expect from "LLM-in-the-loop" methods.

A fuller picture (numbers from our setup where measured, otherwise approximate for the comparable production stack):

	EDP / Bayesian-EDP	LinTS-warm	CombLinUCB	Neural ranker (est.)
Training-time compute	~9 subagent calls / run	per-session SGD	per-session SGD	hours–days of GPU
Per-decision serving cost	~1 ms CPU (245 numbers)	~1 ms CPU	~5 ms CPU (slate UCB)	~20 ms GPU
External deps at serving	0	0	0	≥,1 model server
Sessions to cold-start viability	0	several K	several K	≥ 10K
Audit infrastructure	inline (curves + reasons in repo)	external	external	external (SHAP/LIME approx.)
Editable by a non-ML reviewer	yes (PR diff)	no	no	no
Closed under LLM edit	yes (well-typed scalar)	partial (only hyperparams)	partial	no

The training-time row is where production budgets are most often surprised. A "neural ranker" deployment that retrains weekly on logged engagement consumes orders of magnitude more compute than the entire LLM-edit loop here, and the resulting weights are not editable by anyone, not even the team that trained them. The serving-time row decides whether the policy can sit inline in a latency-budgeted page render; the dependency row decides whether it survives a model-server outage. EDP wins both rows by construction. The "sessions to cold-start viability" row is the deployment property §5.7 quantifies directly.

7. Limitations

Four limitations genuinely bind the conclusions of the paper. The remaining caveats (no formal regret bounds, no neural-bandit baselines, no real deployment, etc.) are noted in line where they apply but do not change any headline.

1. **Ground-truth circularity.** `TRUE_NEEDS` and `TRUE_PROVISIONS` are Claude-authored, the editor at each checkpoint is a Claude subagent, and the diagnostic report it reads is computed from rewards generated by those Claude-authored maps. The §2.2 misalignment between EDP's internal `F`-code taxonomy and the 7-need ground truth, and the Appendix C adversarial-perturbation check (±,15 % per provision, K=5 seeds; the 2× EDP-vs-bandit gap survives), together rule out the worst case where the win is an artefact of the exact numbers Claude chose. They do not rule out a systematic LLM bias correlating ground truth and editor. The right experiment is cross-LLM regeneration of `TRUE_PROVISIONS` with GPT-4 or Gemini; we have no access to those APIs in this pipeline and list this as the largest validity item open.
2. **Capability claims that depend on single trials.** Layer-1 PWL evolution (§5.4d, single trial), Robust-EDP wrapper (§5.8, single rep), structural exploration with a new widget mid-run (§5.9, single trial), and 3-draw ensemble selection (§5.10, K=3) all use too few replicates to support a quantitative claim. Where these sit in the narrative we say "the mechanism works, the value is not yet shown." Multi-seed evaluation (K ≥ 10) of each requires new subagent calls and is the largest empirical item

open. Until then, the demonstrated affordance is *coefficient updates within a fixed policy class plus an extensible edit grammar*. "Evolvable" in the strongest sense (policy-class growth that pays off) remains a capability claim.

3. **Submodular reward is partly what EDP assumes.** Diminishing returns on (need × provision) is the structure adaptive-submodular composition is designed for, and GreedyLinTS (§5.4c) inherits the same structural fit. §5.4e perturbs this with a same-type substitutes penalty and shows the architecture advantage persists (Bayesian-EDP still beats every bandit by 5–15 pp; the gap vs the strongest bandit widens from 3 to 5.5 pp), so the assumption is not load-bearing. Other non-submodular regimes (threshold effects, full bilinear interactions, complementarity) are untested.
4. **Reward-model misspecification in Bayesian-EDP.** The linearised reward predictor `R_hat = a + b * Sum_k s_k(theta)` captures 30–40 % of the noise-free reward variance (Pearson 0.28 parametric, 0.52 LLM). Bayesian-EDP works empirically because the heavy Gaussian regulariser (λ=2.0) keeps the SGD's role local around the LLM-anchored config, but a richer reward model (per-(persona, category) intercept, quadratic interaction on score-sum) is the right replacement for any production deployment. The current Bayesian-EDP margin is small enough on parametric (1.1 pp over EDP-agent in the wrong direction) that the misspecification matters at the magnitude level.

Other scope notes — no formal regret bounds; only LinTS/Slate-LinTS/CombLinUCB on the bandit side (no NeuralUCB, IPS, DR); LLM-as-policy in §5.8 is an asymmetric baseline (one-shot, no reward signal); only one drift mechanism studied; cost analysis is approximate; the audit-trace prose claims of §6.2 use one worked decision and one worked reviewer-edit exchange — are noted where they apply.

8. Future Work

The open directions split into three themes: pushing the *evolvability* of the primitive past coefficient-edit-within-fixed-class; building more *Software 3.0 constructs* on top of the GAM substrate; and the standard list of empirical extensions reviewers will ask for.

8.1 Earning the "evolvable" qualifier

The current paper supports coefficient updates within a fixed policy class. The infrastructure already covers structural moves (Layer-1 PWL shape edits in §5.4d; new-widget addition in §5.9), but the structural results are single-trial and do not yet show a payoff. Three directions push on this directly.

Structural growth multi-seeded with adversarial growth events. The next experiment is K=10 replications of the structural-exploration setup (§5.9), with the catalog event itself drawn from a small distribution of *plausible* new widgets the agent could not have anticipated (a `gift_mode` widget, a `seasonal_promotion` overlay, a `cross-category_bundle`). If the agent integrates the new widget into the policy class within one checkpoint *and* the integrated version measurably outperforms a frozen baseline at K=10, the strongest "evolvable" reading is empirically earned. Until that experiment runs, "Evolvable" remains the capability claim the §1 disclaimer flags.

New shape functions and new problem dimensions. The current Layer-1 has 7 problem codes the LLM authored at design time. A richer evolvability is the agent *proposing a new problem dimension* mid-run (e.g., a `gift_uncertainty` code with its own signal extractors and shape functions) and then editing widget GAM specs to address it. The PRFAQ section on discovery (Axes 1–4: funnel, function, program, event) gives the mechanism but we have no empirical realisation. A natural staging: feed the agent both the current report and a *clustered residual-regret view* (sessions with low predicted problem scores but high regret), and let it emit `{add_problem, add_signal_extractor, add_shape_functions, add_widget_addressing}` as a single coherent edit batch.

Multi-session strategies as Layer-2 code. The PRFAQ describes temporal strategies expressed as conditional code (confidence-before-urgency sequencing, visit-escalation ladders, counterfactual self-correction on past mistakes). These are not in the current paper — every session is treated as i.i.d. A natural extension is a third "Layer 2.5" of symbolic edge-case rules (small Python or DSL snippets the agent emits and a human

approves), which the orchestrator merges into the serving function as user-state-dependent branches. The edit grammar would extend from `{path, from, to, reason}` to `{snippet, gate_condition, reason}`. This is where "intelligence as readable code" reaches its full Software-3.0 form: the system's intelligence is its git log of small typed code edits.

Closed-loop self-criticism. §5.10's negative ensemble result suggests that 3 independent draws plus held-out validation does not reliably select better edits. A more useful self-criticism loop is the *agent reading its own edit and reasoning about the second-order effects* before submitting: "if I raise `easy_returns_promise.on_cov.F46`, what other widgets does that displace in slots 2-4 for `returner_anxious`?" A specialised diagnostic that simulates the post-edit composition on a small held-out slice, then exposes the displacement to the agent in the same checkpoint, is a tractable next step.

8.2 More Software 3.0 constructs on the GAM substrate

The GAM is one Software 3.0 primitive. The deployment story benefits from several adjacent constructs the paper does not yet build.

Compiled policy bundles. The 9 KB JSON policy of Appendix B can be compiled to a self-contained 100-line C function or JS bundle for serverless / edge serving. The edit grammar already binds parameters to named paths; codegen from the JSON to the served binary is mechanical. This makes "the policy is its git log" a literal property at the deployment layer.

LLM-authored widget content + retriever configs as adjacent V3 artifacts. Each widget's retriever (the Solr query, the cached content, the explanation template) is also an LLM-authored, human-reviewable code artifact, with a similar atomic edit grammar (`{retriever, field, from, to, reason}`). Co-evolving the GAM scoring with the retriever configs would extend the "primitive + closure" claim from the orchestration layer to the content layer. The PRFAQ Q2.3 sketches this; we have not built it.

The agent's own prompt as a versioned artifact. Right now the diagnostic-report template is a hand-written Markdown skeleton. The natural next step is to *let the agent edit its own diagnostic report's template at long-cadence checkpoints* — adding new sections when it discovers a new failure mode it cannot diagnose with the current report. This is a meta-evolution loop: the policy program *and* the report-format that drives the policy program both grow under agent edits with human review. It generalises the §5.4 finding that the structured report is the carrier of the gain.

Cross-surface policy sharing. The current EDP serves one surface (the PDP). The PRFAQ Q8.1 lists five more (search, browse, home feed, cart, post-purchase email). Each has its own widget registry but shares the 7-problem Layer-1 taxonomy and the same edit grammar. Multi-surface deployments would let the agent share *Layer-1 edits across surfaces* (since signal-to-problem mappings are universal) while keeping *Layer-2 edits per surface* (since widget semantics differ). This is the simplest interesting structural test of the primitive: does a single agent operating across 6 page surfaces produce more value than 6 independent agents, and does the Layer-1 sharing accelerate convergence?

Stakeholder-authored edits. The current edit grammar is LLM-emitted. The same grammar admits *PM-emitted edits*: a non-ML stakeholder opens a PR with `{widget: returns_explainer, path: base, to: 0.4, reason: "January return season; surface earlier"}`. This is a real consequence of the closure property and is mechanical to support; the experiment is whether stakeholder-authored edits survive AB testing at rates comparable to agent-authored ones. If they do, the primitive's value is not the agent — it is the *substrate that makes both humans and agents productive editors of the same artifact.*

8.3 Standard empirical extensions

Neural bandits. NeuralUCB and small-MLP + Thompson sampling have richer policy classes than linear models and might recover some of the lab-condition gap. Page-level attribution is still the dominant production stressor, and we predict neural methods do not close the production gap (the credit-assignment problem is structural, see §5.2 and §5.3), but a direct experiment would settle it.

Counterfactual estimators. IPS and Doubly Robust estimators can in principle recover partial per-slot credit if a propensity model is available. These methods need their own exploration policy and a logging policy; integrating them is a non-trivial extension and a separate study.

Cross-LLM ground truth. The largest validity item open (§7.1). Regenerate `TRUE_PROVISIONS` and `TRUE_NEEDS` with GPT-4 and Gemini in addition to Claude, rerun the headline comparisons. Resolves the ground-truth-circularity threat in a way the adversarial perturbation only partially addresses.

Layer-1-aware diagnostics + interleaved checkpoints. §5.4d added the Layer-1 PWL-edit capability and observed no single-trial improvement. Two follow-ups: (a) interleave Layer-1-only and Layer-2-only checkpoints to attribute marginal value, (b) add Layer-1-specific diagnostics to the report (e.g., "X% of high-regret sessions had problem F32 detected at <0.3 despite N1_fit > 0.7"). Per-(persona, category) conditional shape functions are a structural step beyond and a separate future item.

Larger ensembles + better validation slicing. §5.10's 3-draw ensemble was statistically indistinguishable from a single draw. Two extensions: K=10–20 draws per checkpoint, and replace the held-out validation slice with a stratified set spanning all (persona, category) cells the live stream is about to encounter.

Live engagement validation. Validate the synthetic-ground-truth ranking against logged engagement data from a deployed system. This is the limitation reviewers will press hardest on; the right way to address it is a logged-eval study, not a richer simulator.

Drift recovery via agent-triggered checkpoints. §5.9's drift test used a fixed checkpoint schedule. Instrument the diagnostic report with a drift detector (persona-mixture tracking, per-cell regret time-series tests) that triggers an unscheduled agent call when the distribution shifts. Combined with §5.9's structural-exploration mechanism, this closes the loop on production non-stationarity.

9. Conclusion

Per-slot LinTS is the wrong abstraction for slate-with-submodular-reward problems under page-level reward; the framing was always misapplied. The EDP architecture (adaptive submodular over a shared GAM parameterisation) is the right baseline, and most of the production-stack advantage we measure comes from that architecture, not from the LLM. GreedyLinTS, the EDP architecture with no LLM prior and pure SGD, already closes 6 pp of the lab-vs-prod gap that per-slot LinTS suffers (12.8 % parametric, 13.3 % LLM in production). The LLM agent and continuous SGD then trade off depending on the simulator: on parametric, EDP-agent (LLM checkpoint edits, no SGD) is the best method we measured (6.5 ± 0.2 %), because the agent's edits are well-targeted; on LLM-persona, Bayesian-EDP (LLM-anchored prior + continuous regularised SGD) is the best (11.0 ± 0.2 %), because the wider persona / category space leaves enough numerical-calibration headroom for SGD to help. Neither method is uniformly best; they are simulator-conditional. The OPRO ablation isolates the structured diagnostic report (not the LLM's general intelligence) as the carrier of the LLM-side gain: cleanly so on the LLM simulator (≈,2.2 σ separation at K=3-4), and weakly so on parametric (≈,1 σ separation, underpowered). The decomposable claim that survives is this. Under page-level reward, use the adaptive-submodular policy class. If you have an LLM with knowledge of the problem domain, use it as a prior and checkpoint editor. If your simulator is rich enough that the LLM's discrete edits don't saturate, add continuous regularised SGD on top.

Appendix A: Open-box decision trace

This appendix walks through Fig. 9 in prose. The session is a `size_anxious_new` persona sampled from the parametric simulator at seed 42, browsing the `dresses` category. The whole trace lives in $14 + 7 + 22 = 43$ named numbers; everything below was generated by reading the policy's internal state, not by any post-hoc interpretability library.

Step 1 — raw signals (14 numbers). The persona's distribution puts most of its mass on three signals: `size_chart` (0.62), `size_conf` (0.18, so `size_conf_inv = 0.82`), and `return_hist` (0.41). Tab-switch is

moderate (0.31), zoom and price-dwell are low. The product-side signal `price_norm = 0.55` (mid-priced item).

Step 2 — Layer-1 problem fingerprint (7 numbers). The PWL curves in Fig. 8 are evaluated at the signal values above, weighted, and clipped to [0, 1]. The fingerprint comes out concentrated on F3.2:

Problem	Score	Driving signal(s)
F3.2 size anxiety	**0.74**	size_chart 0.62 → 0.61; size_conf_inv 0.82 → 0.78; return_hist 0.41 → 0.30
F4.6 return hesitation	0.31	return_hist 0.41 → 0.30
F4.1 comparison	0.18	tab_switch 0.31 → 0.16
F3.3 / F4.3 / F4.5 / F5.1	< 0.10	(signals near baseline)

The fingerprint is a 7-d vector with one dominant axis. Layer-2 will see this and compose accordingly.

Step 3 — Layer-2 slot-1 scoring (22 numbers per slot). For slot 1, remaining = the fingerprint above, coverage = 0. The score decomposes as `base + Sum on_rem[p]*remaining[p] + Sum on_cov[p]*coverage[p] - slot_decay*0`. The top five candidates (Fig. 9, bottom panel):

Widget	base	on_rem contribution	on_cov	total
`fit_reassurance`	0.10	$2.2 \times 0.74 + 0.6 \times 0.31 =$ **1.81**	0	**1.91**
`size_guide`	0.05	$2.0 \times 0.74 =$ **1.48**	0	**1.53**
`low_return_alts`	0.05	$1.0 \times 0.74 + 1.8 \times 0.31 = 1.30$	0	1.35
`easy_returns_promise`	0.10	$1.2 \times 0.31 = 0.37$	0	0.47
`similar_items`	0.45	small	0	0.46

`fit_reassurance` wins because of the F3.2 on_rem slope, not because of a high base. After it is placed, F3.2 remaining drops by 0.55 (its addr value) to 0.19, F3.2 coverage rises by 0.55 to 0.55, and slot 2 scoring reruns with the updated state. `size_guide`, with `on_cov.F32 = -1.4` in the LLM prior, gets *penalised* in slot 2 by $1.4 \times 0.55 = 0.77$ — the explicit anti-synergy stops the page from wasting a second slot on fit guidance.

(The round-2500 agent edit batch in Fig. 10 mutates this exact anti-synergy on `size_guide.on_cov.F32` from $-$,1.4 to +0.6, converting it from a substitute to a chained complement for the size-anxious persona. The reason field is preserved verbatim. Whether the edit pays off is empirical (§5.4b–5.4c): it does on parametric, less clearly so on LLM.)

What an interpretability reviewer can verify in 5 minutes. That the size-anxious persona's page leads with `fit_reassurance` because of the F3.2 on-remaining slope; that the second slot's identity is sensitive to the size_guide / fit_reassurance synergy parameter; that swapping the persona changes the fingerprint, the slot-1 winner, and the slot-2 anti-synergy — all by editing values the reviewer can see and write. No SHAP, no LIME, no post-hoc surrogate. The decision trace is the policy.

What an agent can do at the same place. The agent reads exactly the table above (plus regret aggregates) and proposes scalar edits to any of the 43 numbers, each with a free-text reason. Fig. 10 shows what one such batch looks like in practice. The unit of work is a number-with-a-reason, not a weight update.

This is the open-box property the paradigm hinges on. Every other claim — cold-start advantage, audit-trail compatibility, LLM-as-prior fusion — derives from being able to do this on every session.

Appendix B: Deployment walkthrough

This appendix sketches how the primitive would embed in a production page-render path, so the deployment claims of §1 and §6.3 have a concrete artifact to discuss.

Step 1 — the policy ships as a JSON file. The 245-parameter learnable policy plus the metadata needed for serving (problem labels, widget descriptors, edit-grammar version) is approximately 9 KB of JSON. It is checked into the same repository as the application code. There is no separate model registry, no model server, no feature store.

```
// policy_v_017.json  (9.2 KB)
{
  "version": "017",
  "approved_by": "alice@team.example",
  "from_round": 7500,
  "shapes": { "F32": { "size_chart": {"bps": [0, 0.2, 0.5, 0.8, 1],
                                      "vals":[0, 0.1, 0.45, 0.75, 0.95],
                                      "weight": 0.45 }, ... }, ... },
  "modules": { "fit_reassurance": {"base": 0.10, "addr": {"F32": 0.55, ...},
                                   "on_rem": {"F32": 2.2, ...},
                                   "on_cov": {"F32": -1.2}, ... }, ... }
}
```

Step 2 — the serving function is ~80 lines of Python. `score_problems`, `score_module`, and `compose` from `edp/policies/edp.py` are the entire decision logic. Given a session feature vector and a category, they return a 6-widget page. No external calls. The same code path runs in CI tests.

```
# Production serving path (sketch)
from edp.policies.edp import compose
import json

POLICY = json.load(open("policy_v_017.json"))

def render_page(session_features, category):
    page = compose(session_features, POLICY["shapes"], POLICY["modules"])
    # page is ['fit_reassurance', 'low_return_alts', ...]
    return [fill_widget(w, category) for w in page]
```

`fill_widget(name, category)` pulls cached content from the widget retriever output — already a production capability today. The EDP layer adds about 1 ms to the page render. The 9 KB policy fits in any inline cache and is loaded at process start.

Step 3 — the audit log is the policy repository's git log. Each agent edit batch is committed as a PR. The PR body is the diagnostic report the agent read plus the edit batch with its reasons. A reviewer reads ten lines and either merges or requests changes (the reviewer-rejection vignette in §6.2 is the realistic shape of one round). The serving policy file is updated by merging the PR; there is no separate deployment.

```
$ git log --oneline policy_v_*.json
b3f2a1c policy v017: round-7500 edits -- strengthen F46 synergy chain (alice)
8d7e120 policy v016: round-5000 edits -- cut over-firing on outfit_completion (alice)
3a9c2f5 policy v015: round-2500 edits -- revive dead returns/size widgets (bob)
0001abc policy v014: initial LLM-authored config from shopping psychology (LLM)
```

A new team member reads the git log to understand what the system has learned. A regulator subpoenas the same log and reads the diagnostic reports and reasons. A non-ML PM proposes a manual edit by opening a PR. None of these workflows require ML infrastructure.

Step 4 — Bayesian-EDP adds a small async loop. If the deployment uses Bayesian-EDP, a separate process drains the delayed-reward queue and SGD-updates the policy file every hour or so. The SGD code is

~50 lines (`edp/policies/bayesian_edp.py`). The resulting numeric drift relative to the LLM prior is logged and bounded; if the SGD wants to move a parameter by more than (say) 50 % of its prior value, the move is held for a human checkpoint review (current implementation is unbounded; bounded drift is a one-line addition).

Step 5 — checkpoint cadence is operational. Every K sessions (we use K = 2,500 in experiments; production probably 50K–500K depending on traffic), the orchestrator builds the diagnostic report and invokes the agent. This is an offline cron job, not a serving-path dependency. If the agent is unavailable, the previous policy keeps serving indefinitely; the system degrades gracefully.

What this isn't. This is a *walkthrough*, not a production case study. We have not deployed this stack at Zalando-scale traffic; the production-scale follow-up is left as the next concrete experiment. The artifact-level claims (policy ships as JSON, audit log is git log, no model server) are mechanical consequences of the primitive's properties; the latency claim (~1 ms) is measured in-process but not in a production rendering pipeline. The cold-start, audit, and serving-cost claims are first-principles. The benchmark claim is empirical, not a deployment result.

Appendix C: Adversarial-perturbation validity check

We cannot run a true cross-LLM check (no GPT-4 / Gemini access in our pipeline). The next-best validity check is **adversarial perturbation of the LLM-authored `TRUE_PROVISIONS`**: each provision value is multiplied by `uniform(1-eps, 1+eps)` for $\epsilon = 0.15$ and clipped to [0, 1], with the perturbation seeded independently each replicate. The greedy oracle is re-computed under the perturbed provisions; methods are scored as % of perturbed-oracle reward lost.

Method	Canonical (no perturbation)	Adversarial (ϵ=0.15, K=5)	Range
EDP-static	10.7 %	**8.9 ± 0.3 %**	8.1 – 9.7 %
EDP-agent	6.5 ± 0.2 %	**7.0 ± 0.2 %**	6.4 – 7.5 %
Bayesian-EDP	7.6 ± 0.3 %	**6.5 ± 0.3 %**	5.9 – 7.2 %
CombLinUCB-warm	10.5 ± 0.1 %	**12.2 ± 0.3 %**	11.4 – 13.1 %
LinTS-warm	18.9 ± 0.1 %	**21.7 ± 0.2 %**	21.4 – 22.4 %

Three findings:

1. The EDP-vs-bandit gap survives the perturbation. EDP variants stay in the 6.5–9 % band; the strongest bandit (CombLinUCB) is in the 11.4–13.1 % band. The roughly 2× margin holds at every seed.
2. The bandits degrade slightly more under perturbation than EDP does (CombLinUCB +1.7 pp, LinTS +2.8 pp; EDP-agent +0.5 pp). The bandit's per-session SGD is fitting the noisy perturbed reward signal; EDP's policy class is closer to canonical and absorbs the perturbation more cleanly.
3. Bayesian-EDP becomes the best method under perturbation (6.5 %, vs EDP-agent at 7.0 %). The SGD's bias-variance trade-off plays differently when the canonical reward is itself jittered: the regularised drift away from the LLM prior is small enough to absorb perturbation noise and flexible enough to track the new optimum.

The check rules out the worst case where EDP's win is an artefact of the exact numerical values the LLM chose for `TRUE_PROVISIONS`. A ±,15 % perturbation is a meaningful chunk of the [0, 1] provision range; if the win were knife-edge, it would not survive.

It does not rule out a systematic LLM bias. Both `TRUE_PROVISIONS` and the editor are Claude-family. A perturbation that randomly noises individual provision values is structurally different from a different LLM family generating systematically different patterns of provision (e.g., a different LLM might think `outfit_completion` strongly addresses N6_trust, not just N5_styling). The true cross-LLM check remains the right experiment for future work, and we list it explicitly in §7 and §8.

For reproduction commands, code structure, and the interactive demos, see the supplementary material (`supplementary.md`).